



**A QUANTIZED LLM DRIVEN FRAMEWORK FOR
AUTOMATED SOFTWARE TESTING**

Lappeenranta-Lahti University of Technology LUT

Master's Programme in Software Engineering and Digital Transformation, Master's thesis
2026

Rashed Ahmed

Supervisor and first examiner: Associate Professor Azeem Akbar

ABSTRACT

Lappeenranta-Lahti University of Technology LUT

LUT School of Engineering Sciences

Software Engineering

Rashed Ahmed

A Quantized LLM Driven Framework for Automated Software Testing

Master's thesis

2026

96 pages, 4 figures, 16 tables and 3 appendices

Supervisor and first examiner: Associate Professor Azeem Akbar

Keywords: large language models, software testing, automated test generation, LLM-assisted testing, test validation, software quality assurance

Unit testing remains one of the most expensive recurring activities in software development, especially in small teams where test maintenance competes with feature delivery. Although automated test generation has improved, many approaches remain difficult to operationalize in everyday workflows because outputs are brittle, execution reliability is inconsistent, and cloud-dependent inference introduces confidentiality and reproducibility concerns. This thesis addresses that gap by implementing and evaluating a local, reproducible framework for LLM-assisted unit test generation for Python repositories.

The framework combines repository file discovery, sandboxed module isolation, prompt-driven test generation with a fine-tuned and quantized Qwen2.5-Coder-1.5B model, AST-based post-processing, pytest execution under Coverage.py, and iterative refinement using error feedback. The implementation is available in both command-line and web-based forms, with persistent run artefacts, per-file reports, and explicit handling of difficult cases through skip and mark-complete controls.

Empirical evaluation is based on eight recorded runs and 22 non-skipped file-level evaluations. The mean per-file statement coverage is 93.82%, with median 100%, minimum 83%, and maximum 100%. The strict pytest pass rate is 81.8% (18/22), while operational completion reaches 90.9% (20/22) when marked-complete outcomes are included. Refinement is required for 4/22 files, with a mean of 0.27 refinement attempts. Results show that the framework can reliably bootstrap compact executable tests on small modules in an offline setting. The main remaining limitation is oracle-level assertion mismatch rather than syntax or import failure, indicating that future improvements should prioritize semantic guidance and stronger correctness signals during refinement. The workflow supports transparent replay, inspection, and iterative configuration tuning.

TIIVISTELMÄ

Lappeenrannan–Lahden teknillinen yliopisto LUT

LUT Teknis-luonnontieteellinen

Software Engineering and Digital Transformation -maisteriohjelma

Rashed Ahmed

Kvantisoitu LLM-pohjainen kehys automatisoidulle ohjelmistotestaukselle

Diplomityö

2026

96 sivua, 4 kuvaa, 16 taulukkoa ja 3 liitettä

Ohjaaja ja tarkastaja: Apulaisprofessori Azeem Akbar

Avainsanat: suuret kielimallit, ohjelmistotestaus, automatisoitu testien generointi, LLM-avusteinen testaus, testien validointi, ohjelmiston laadunvarmistus

Ohjelmistotestaus on keskeinen ohjelmistojen laadun ja luotettavuuden varmistamisessa, mutta yksikkötestien kirjoittaminen ja ylläpito on edelleen aikaa vievää. Perinteiset automaattiset menetelmät voivat parantaa kattavuutta, mutta niiden tuottamat testit eivät aina ole suoraan käytännössä hyödynnettäviä. Myös kielimallipohjaisessa testigeneroinnissa on haasteita, kuten suoritettavuusongelmat, epävarmat odotusarvot sekä toistettavuus- ja tietosuojaoingelmat pilvipohjaisissa ratkaisuuissa. Tässä diplomityössä tarkastellaan, miten Python-projekteille voidaan rakentaa käytännöllinen, paikallisesti toimiva ja toistettava LLM-avusteinen testigenerointiprosessi.

Työssä toteutetaan päästä päähän -kehys, joka etsii lähdetiedostot, generoi PyTest-testit hienosäädetyllä ja kvantisoidulla Qwen2.5-Coder-1.5B -mallilla, validoi ja normalisoi tulokset AST-pohjaisilla tarkistuksilla, suorittaa testit Coverage.py-mittauksella eristetyssä hiekkalaatikossa sekä iteratiivisesti jalostaa epäonnistuvia testejä pytest-palautteen perusteella. Kehys on toteutettu sekä komentoriviputkena että Flask-pohjaisena verkkosovelluksena, joka tallentaa lokit, välitulokset ja tiedostokohtaiset raportit.

Arviointi perustuu kahdeksaan tallennettuun ajokertaan ja 22 ei-ohitettuun tiedostotason arviointiin. Tiedostokohtainen statement-kattavuus on keskimäärin 93.82% (mediaani 100%, minimi 83%, maksimi 100%). Tiukan pytest-kriteerin mukainen läpäisyosuus on 81.8% (18/22). Kun marked-complete-tilat huomioidaan, operatiivinen valmistumisaste nousee 90.9%:iin (20/22). Jalostusta tarvittiin 4 tiedostossa 22:sta, ja jalostuskiirroksia oli keskimäärin 0.27 tiedostoa kohden. Tulokset osoittavat, että kehys tuottaa paikallisesti ajettavia, kompakteja testejä tehokkaasti erityisesti pienille Python-moduuleille. Keskeinen jäljelle jäävä rajoite liittyy odotusarvojen semanttiseen oikeellisuuteen, ei niinkään syntaksi- tai tuontivirheisiin.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my supervisor, Associate Professor Azeem Akbar, for his continuous guidance, thoughtful feedback, and consistent encouragement throughout this thesis. His academic leadership in software engineering and software process improvement, together with his broader research contributions in artificial intelligence, data-driven methods, and large language model applications, provided both direction and inspiration for this work. His high scholarly standards and practical perspective significantly strengthened the quality of this study.

I also gratefully acknowledge LUT University, LUT School of Engineering Sciences, and the Master's Programme in Software Engineering and Digital Transformation for providing an excellent research environment in Lappeenranta. The institutional support, academic resources, and collaborative atmosphere were essential for completing this thesis.

My thanks are also extended to colleagues and peers who shared constructive discussions and valuable suggestions during different phases of the project. Finally, I am deeply thankful to my family and close friends for their patience, trust, and unwavering support, which sustained me throughout this journey.

SYMBOLS AND ABBREVIATIONS

This section summarises the principal symbols, parameters, acronyms, and abbreviations used throughout the thesis. The symbols are aligned with the notation introduced in the evaluation and experimental chapters, while the acronym list reflects the terminology used in the framework design, implementation, and discussion.

Technical Symbols and Parameters

Table 1: Symbols and mathematical/parameter notation used in the thesis.

Symbol	Definition	Context of Use
S_{tot}	Total number of executable statements in the module under analysis.	Statement coverage computation.
S_{miss}	Number of executable statements not exercised by generated tests.	Statement coverage computation.
S_{cov}	Number of covered executable statements, where $S_{\text{cov}} = S_{\text{tot}} - S_{\text{miss}}$.	Statement coverage computation.
C_{stmt}	Statement coverage percentage, $C_{\text{stmt}} = \frac{S_{\text{cov}}}{S_{\text{tot}}} \times 100\%$.	File-level and aggregate effectiveness metric.
N_{tested}	Number of files evaluated in a run (excluding non-evaluated files).	Run-level aggregation formulas.
N_{pass}	Number of files with a strict pytest pass outcome.	Pass-rate and completion-rate metrics.
N_{marked}	Number of files marked complete through operator decision after exhausted refinement.	Operational completion metric.
N_{skip}	Number of files skipped by user control action.	Run-state categorisation.
N_{skipfail}	Number of files skipped due to repeated failures after refinement budget is exhausted.	Failure/skip analysis.
R_{pass}	Strict pytest pass rate across evaluated files.	Autonomous correctness indicator.

Symbol	Definition	Context of Use
R_{complete}	Completion rate including both strict passes and marked-complete outcomes.	Operational usability indicator.
A_i	Refinement attempts used for file i .	Iterative feedback analysis.
$\bar{A}$	Mean refinement effort across evaluated files.	Aggregate refinement metric.
$P(A_i > 0)$	Fraction of files requiring at least one refinement iteration.	Stability of first-shot generation.
T	Number of generated test functions (recorded as raw and final counts).	Test suite size analysis.
r	LoRA rank hyperparameter used in parameter-efficient fine-tuning.	QLoRA configuration.
α	LoRA scaling factor (alpha) used in adapter training.	QLoRA configuration.
n_ctx	LLM context window size in tokens.	Inference configuration in llama-cpp-python.
n_threads	Number of CPU threads used for local model inference.	Inference performance configuration.
n_batch	Token batch size used by the inference runtime.	Inference throughput configuration.
-maxfail=1	Pytest fail-fast parameter that stops execution after the first failure.	Fast feedback in refinement loop.
COVERAGE_FILE	Environment variable specifying per-run/per-file coverage data location.	Coverage.py subprocess isolation.

Acronyms and Abbreviations

Table 2: Acronyms and abbreviations used in the thesis.

Abbreviation	Full Form	Description in This Thesis
LLM	Large Language Model	Neural language model used to generate and refine unit tests from source code context.

Abbreviation	Full Form	Description in This Thesis
AST	Abstract Syntax Tree	Python code representation used for syntactic checks, test normalization, and structural transformations.
SBST	Search-Based Software Testing	Optimization-based family of automated test generation techniques used as baseline context in literature positioning.
PyTest	Python Testing Framework	Test runner and assertion framework used for executing generated tests.
API	Application Programming Interface	Interface boundary, including LLM invocation APIs and web control endpoints.
CLI	Command-Line Interface	Non-web execution mode provided by <code>run_pipeline.py</code> .
UI	User Interface	Browser-based interaction layer provided by the Flask dashboard.
JSON	JavaScript Object Notation	Structured format for per-file and run-level reports and artefact metadata.
HTML	HyperText Markup Language	Output/report format used for web dashboard rendering and coverage report viewing.
SQL	Structured Query Language	Query language used through SQLite-backed run tracking.
SQLite	SQLite Database Engine	Embedded relational store used for run history, logs, file results, and test artefacts.
CPU	Central Processing Unit	Local inference hardware target for the quantized model.
GPU	Graphics Processing Unit	Hardware used during model fine-tuning in cloud environment.
VRAM	Video Random Access Memory	On-device memory capacity used to characterize fine-tuning hardware.

Abbreviation	Full Form	Description in This Thesis
QLoRA	Quantized Low-Rank Adaptation	Parameter-efficient fine-tuning approach combining low-rank adapters with quantized base weights.
LoRA	Low-Rank Adaptation	Adapter-based method for updating a small subset of trainable parameters.
PEFT	Parameter-Efficient Fine-Tuning	Family of fine-tuning strategies that update a limited parameter subset.
GGUF	GPT-Generated Unified Format	Model serialization format used by <code>llama.cpp/llama-cpp-python</code> for local inference.
NF4	NormalFloat4	4-bit quantization scheme used during model fine-tuning workflow.
ChatML	Chat Markup Language	Message-format template used to structure instruction-style model prompts.
CI/CD	Continuous Integration / Continuous Delivery	Development workflow context discussed for future integration of the framework.
ICLR	International Conference on Learning Representations	Venue referenced for the TestGenEval benchmark publication context.
SWE-Bench	Software Engineering Benchmark	Source benchmark from which TestGenEval is derived.

TABLE OF CONTENTS

ABSTRACT	ii
ACKNOWLEDGMENTS	iv
SYMBOLS AND ABBREVIATIONS	v
1 INTRODUCTION	8
1.1 Research Problem and Motivation	9
1.2 Research Objectives	9
1.3 Contributions	10
1.4 Thesis Structure	10
2 BACKGROUND	11
2.1 Software Testing Principles	11
2.2 Program Representation and Lightweight Code Analysis	12
2.3 Large Language Models for Code and Domain Adaptation	12
2.4 Prompt Engineering and Iterative Refinement	12
2.5 Evaluation Signals: Executability, Coverage, and Effectiveness	13
3 LITERATURE REVIEW	14
3.1 Automated Unit Test Generation: Core Paradigms and Practical Trade-offs .	14
3.1.1 Symbolic execution as a foundation for systematic exploration . . .	14
3.1.2 Concolic testing: pragmatic symbolic reasoning grounded in concrete runs	15
3.1.3 Random and property-based testing: low-infrastructure generation with strong oracles	15
3.1.4 Feedback-directed random testing and regression-style assertions . .	16
3.1.5 Search-based test generation and assertion synthesis	17
3.2 Adequacy, Coverage, and the Test Oracle Problem	17
3.2.1 Coverage as a partial and sometimes misleading indicator	17
3.2.2 The oracle problem as the dominant barrier to full automation . . .	18
3.3 Mutation Testing and Stronger Adequacy Signals	18
3.3.1 Mutation testing as a complement to coverage	18
3.4 LLMs for Code and Test Generation	19
3.4.1 LLMs as code generators and the role of prompting	19
3.4.2 Execution-based evaluation and its mirror image in test generation .	19

3.5	LLM-Based Unit Test Generation: Empirical Findings and System-Level Lessons	19
3.5.1	Evaluating ChatGPT-style generation and improving reliability . . .	19
3.5.2	Comparing LLM generation with SBST baselines	20
3.5.3	Evaluation methodology for LLM unit test generation	20
3.5.4	Mutation-guided and signal-guided LLM test generation	20
3.6	Synthesis and Research Gaps Motivating This Thesis	21
3.7	Comparison of Related Work	21
4	PROPOSED FRAMEWORK	28
4.1	Purpose and Motivation	28
4.2	High-Level System Architecture	29
4.3	Framework Pipeline: Stage-by-Stage Design	30
4.3.1	Stage 1: Input and Code Analysis	30
4.3.2	Stage 2: Prompt Engineering	31
4.3.3	Stage 3: LLM-Based Test Generation	32
4.3.4	Stage 4: Validation and Filtering	33
4.3.5	Stage 5: Test Execution, Coverage Analysis, and Iterative Feedback	33
4.4	Workflow Architecture and Component Interaction	34
4.5	Sequence of Operations	35
4.6	Data Flow Through the System	36
4.7	Technical Innovations and Design Rationale	37
5	IMPLEMENTATION	39
5.1	Implementation Scope and Module Structure	39
5.2	Technology Choices and Runtime Dependencies	39
5.3	Model Adaptation and Packaging Workflow	40
5.4	Inference Layer Implementation	40
5.4.1	Model Loading and Invocation	40
5.4.2	Prompt Templates	41
5.4.3	Raw Output Normalization	41
5.5	Repository Orchestration	41
5.5.1	Run Directory Layout	42
5.5.2	Discovery and File Selection	42
5.5.3	Per-file Processing Loop	42
5.5.4	Decision Handling After Refinement Exhaustion	42
5.6	Execution and Coverage Measurement	42
5.7	Web Application and Persistence Layer	43
5.8	Command-line Execution Path	43

5.9	Implementation Challenges and Engineering Responses	44
5.10	Summary	44
6	EXPERIMENTAL SETUP	45
6.1	Dataset: TestGenEval	45
6.1.1	Dataset Origin and Composition	45
6.1.2	Rationale for Dataset Selection	46
6.2	Data Preprocessing	47
6.2.1	Comment Stripping	47
6.2.2	Length Clipping	47
6.2.3	Training Text Formatting	47
6.2.4	Tokenization and Label Masking	47
6.3	Experiment Environment	47
6.3.1	Fine-Tuning Environment	48
6.3.2	Inference Environment	48
6.4	Experiment Design	49
6.4.1	Evaluation Procedure	49
6.4.2	Evaluation Metrics	49
6.4.3	Controls and Baselines	49
6.5	Model and Pipeline Configuration Parameters	49
6.5.1	Fine-Tuning Hyperparameters	49
6.6	Post-Quantization Model Verification	52
6.7	Reproducibility Considerations	52
6.8	Summary	52
7	EVALUATION METRICS	53
7.1	Measurement Context	53
7.2	Coverage Metrics	54
7.2.1	Per-file Statement Coverage	54
7.2.2	Run-level Overall Coverage	54
7.3	Execution Outcome Metrics	55
7.3.1	Strict Pass and Operational Completion	55
7.3.2	Skip and Failure Rates	55
7.4	Refinement Metrics	56
7.5	Test Suite Size Metrics	56
7.6	Structural Quality Signals	56
7.7	Metric Summary	57
8	RESULTS AND ANALYSIS	58

8.1	Overview of Recorded Experimental Runs	58
8.2	Aggregate Performance Across All Files	59
8.3	Coverage Outcomes and Interpretation	60
8.3.1	Why overall coverage appears as 89% vs. 94%	60
8.3.2	Per-file coverage patterns	60
8.4	Executability and Correctness Outcomes	61
8.4.1	Strict pass rate vs. operational completion	61
8.4.2	Nature of failures: oracle errors dominate	62
8.5	Effect of Iterative Refinement	63
8.5.1	When refinement helps	63
8.5.2	When refinement does not help	63
8.5.3	Refinement budget and development variability	63
8.6	Test Suite Size and Structure	64
8.7	Observed Trends, Improvements, and Unexpected Behaviours	64
8.8	Errors, Failures, and Limitations	65
8.8.1	Limitations of the recorded experiment set	65
8.8.2	Technical limitations of LLM-generated tests	66
8.9	Summary	66
9	DISCUSSION	67
9.1	Interpretation of Effectiveness and Reliability	67
9.2	Comparison with Expectations and Prior Approaches	68
9.3	Strengths Demonstrated by the Framework	68
9.4	Limitations and Likely Causes of Performance Gaps	69
9.5	Practical Implications for Real-World Testing	70
10	CONCLUSIONS	71
11	FUTURE WORK	73
12	REFERENCES	74
APPENDICES		
A	Prompts Used	
A.1	Notebook Prompts for Fine-Tuning and Evaluation	
A.1.1	Notebook Prompt A: Training Text Construction	
A.1.2	Notebook Prompt B: Evaluation Prompt Construction	
A.1.3	Notebook Prompt C: System Message Used in Chat Template	
A.2	Runtime Prompts Used by the Main Pipeline	

- A.2.1 Runtime Prompt 1: Test Generation Template
 - A.2.2 Runtime Prompt 2: Test Refinement Template
- A.3 Additional Script-Level System Instructions
 - A.3.1 Script System Instruction A: Generation
 - A.3.2 Script System Instruction B: Refinement

B Sample Generated Tests

- B.1 Sample A: Leap-Year Function (Normal and Edge Cases)
- B.2 Sample B: Factorial Function (Normal and Boundary Input)
- B.3 Sample C: Simple Arithmetic (Normal and Error Handling)
- B.4 Sample D: Refined Output After Feedback Rounds

C Additional Results

- C.1 Supplementary Run-Level Completion and Test-Volume Detail
- C.2 Refinement and Failure-Type Distributions
- C.3 Coverage Denominator Sensitivity (Supplementary View)

LIST OF FIGURES

1	System architecture of the LLM-assisted software testing framework.	29
2	Five-stage pipeline for LLM-assisted unit test generation and refinement. . .	30
3	Workflow architecture and component interactions of the framework.	35
4	Sequence of interactions during test generation and refinement.	36

LIST OF TABLES

1	Symbols and mathematical/parameter notation used in the thesis.	v
2	Acronyms and abbreviations used in the thesis.	vi
3	Comparison of related work and its relevance to the proposed framework. .	22
4	Principal columns of the TestGenEval dataset and their descriptions.	46
5	Composition of the TestGenEval dataset splits.	46
6	Hardware and software specifications for the fine-tuning and inference environments.	48
7	Model inference configuration parameters.	50
8	Pipeline configuration parameters.	50
9	Fine-tuning hyperparameters for the QLoRA training procedure.	51
10	Summary of evaluation metrics and their intended interpretation.	57
11	Run-level summary	59
12	Representative file-level outcomes	61
13	Supplementary run-level detail	
14	Distribution of refinement attempts	
15	Failure-category counts	
16	Coverage denominator pattern	

1 INTRODUCTION

Software testing is a foundational practice for improving software quality, reliability, and maintainability. Unit tests, in particular, provide fast, repeatable checks that support refactoring and regression prevention, yet constructing and maintaining effective unit test suites remains labour-intensive. This tension—between the clear value of testing and the high cost of producing tests at scale—has sustained a long research agenda in automated test generation.

Historically, automated approaches have prioritised structural exploration to reduce the manual effort of crafting tests. Feedback-directed random testing, exemplified by tools such as Randoop, generates sequences of calls and retains those that execute without failure, producing executable tests with minimal specification knowledge [1]. Search-based software testing (SBST) reframes test generation as optimisation, guiding the search towards adequacy criteria such as statement or branch coverage [2]. Tools such as EvoSuite demonstrate that these ideas can be operationalised into practical unit-level workflows that automatically construct test suites and often attain high coverage [3]. However, these approaches also expose persistent limitations when the goal is not merely to execute code, but to validate correct behaviour in a way that developers can trust.

One central limitation is the oracle problem: even when inputs can be generated automatically, determining whether observed outputs are correct is often underspecified, expensive, or ambiguous [4]. Automated techniques can therefore produce tests that execute and increase coverage while still lacking meaningful assertions or encoding brittle expectations. Coverage is a useful proxy for exploration, but it is not a reliable stand-in for fault detection effectiveness, and empirical evidence indicates the correlation can be weak [5]. In practice, developers often reject automatically generated tests not because they fail to run, but because they provide low confidence in behavioural correctness.

In recent years, large language models (LLMs) have created new possibilities for generating code and code-adjacent artefacts from natural-language and program context. Few-shot prompting demonstrated that sufficiently large transformer models can adapt to new tasks from instructions and examples without task-specific training [6]. Work on models trained on code has shown strong capability in program synthesis and completion [7]. These capabilities motivate LLM-based unit test generation because LLMs can directly emit readable test functions and idiomatic assertions that resemble developer-written tests.

Despite this promise, existing LLM-based unit test generation approaches face practical barriers to adoption. Many systems rely on cloud-hosted proprietary models accessed via

APIs, introducing recurring cost, latency, and confidentiality concerns when source code must be transmitted to third-party services. Even strong models can produce non-executable output, omit required imports, or make unfounded assumptions that lead to failing tests and incorrect assertions. Recent evaluations of ChatGPT-style unit test generation underscore these challenges, reporting frequent issues around executability and assertion correctness [8]. These limitations reveal a gap between research demonstrations and tools that can operate reliably within automated workflows.

1.1 Research Problem and Motivation

This thesis addresses the following research problem: *How can an end-to-end framework reliably generate and evaluate executable unit tests for Python projects using an LLM, while operating fully locally to preserve confidentiality and reduce deployment cost, and while providing systematic feedback and artefacts that support developer review?* The problem encompasses the lifecycle required to make generated tests actionable: selecting suitable target files, constructing prompts that reflect the execution environment, validating and normalising model output, executing tests in isolation, measuring outcomes such as coverage, and iteratively refining tests based on observed failures.

The motivation for this work is grounded in a practical observation: developers benefit most from test automation when the produced artefacts are executable, readable, and accompanied by transparent evidence of what was explored. The emphasis on offline execution and privacy-preserving deployment led to a deliberate design choice: use a smaller, open-source code model that can run locally in a quantized format, then compensate by combining strict prompting constraints with an execution-driven refinement loop. The resulting framework treats the LLM as a proposal mechanism and treats execution outcomes and coverage measurements as verifiable signals for acceptance and improvement.

1.2 Research Objectives

The objectives of this thesis are:

- To design and implement a reproducible pipeline for LLM-assisted unit test generation for Python repositories, producing `pytest`-compatible tests that execute in a controlled environment.
- To integrate automated evaluation signals (test executability and structural coverage) to assess generated tests consistently across files and runs.
- To implement an iterative refinement mechanism that uses concrete execution feedback to repair failing tests with minimal human intervention.

- To provide comprehensive artefact logging and reporting that supports transparency, debugging, and comparison between raw and refined generation stages.

1.3 Contributions

The thesis makes the following contributions:

- An end-to-end, repo-level framework that discovers Python source files, generates unit tests with a locally executed LLM, validates outputs, and runs tests under automated orchestration.
- A structured refinement loop that incorporates runtime error feedback to improve executability and stabilise generated tests.
- An evaluation workflow that records per-file and run-level metrics, including statement coverage summaries and persistent artefacts such as logs and HTML coverage reports, enabling reproducible analysis.
- A practical deployment-oriented perspective on LLM-based testing that prioritises offline operation, confidentiality, and transparent reporting as first-class requirements.

1.4 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 (Background) introduces core concepts for LLM-assisted test generation. Chapter 3 (Literature Review) surveys prior work in automated test generation and LLM-based approaches. Chapter 4 (Research Methodology) describes the evaluation approach. Chapter 5 (Proposed Framework) presents the architecture and pipeline stages, and Chapter 6 (Implementation) details the software artefacts that realise the design. Chapters 7–9 define the experimental setup, evaluation metrics, and results analysis, respectively. Chapter 10 (Discussion) interprets findings and limitations, Chapter 11 (Conclusions) summarises the thesis, and Chapter 12 (Future Work) outlines directions for extension.

In summary, this thesis is motivated by the need for practical, trustworthy automation in unit testing and by the emergence of LLMs as a new mechanism for synthesising human-like test code. By grounding generation in local execution, enforcing executable constraints, and systematically recording evaluation artefacts, the proposed framework seeks to narrow the gap between LLM-driven code synthesis and the stringent reliability requirements of software testing practice.

2 BACKGROUND

LLM-assisted test generation combines classical testing principles with modern neural code synthesis. This thesis builds on established ideas in unit testing, coverage-based adequacy, and automated test generation, while recognising that large language models (LLMs) shift practical bottlenecks toward oracle quality and execution validity. Because the framework operates as an end-to-end pipeline that generates, validates, executes, and refines `pytest`-style tests, the key background spans (i) unit tests as behavioural specifications, (ii) coverage as a pragmatic adequacy signal, (iii) AST-based lightweight code checks, and (iv) prompting and parameter-efficient fine-tuning for domain adaptation.

2.1 Software Testing Principles

Unit testing aims to verify the behaviour of individual functions or small modules in isolation. It provides rapid feedback and localises failures, but a unit test is only as useful as its assertions: assertions embody an oracle that distinguishes correct from incorrect behaviour. In practice, the “oracle problem” is pervasive: for many programs the correct output is unknown, underspecified, or expensive to compute [4]. This is especially relevant for automatically generated tests, which may execute successfully and increase coverage while encoding incorrect expectations.

Automated test generation has historically targeted structural adequacy criteria to guide exploration. Random and feedback-directed approaches sample behaviours without deep semantic reasoning, often achieving useful coverage with low analysis overhead [9, 1]. Search-based software testing (SBST) frames input discovery as optimisation over objectives such as statement or branch coverage [2]. Tools such as EvoSuite operationalise this idea at the unit level and emit executable test code [3]. These traditions motivate the evaluation logic in this project: the pipeline treats execution and coverage as measurable, repeatable signals.

Coverage is a pragmatic proxy for exploration, but it is not a direct measure of fault detection. Empirical studies show that high coverage is not necessarily strongly correlated with effectiveness, particularly when tests lack meaningful assertions [5]. The framework therefore records coverage while also treating `pytest` executability and refinement outcomes as first-class signals.

To complement coverage, mutation testing injects small syntactic changes (mutants) and checks whether the test suite detects them [10, 11]. Mutants are not always representative of real faults, but mutation can be a useful indicator when interpreted carefully [12]. In this

project it is an optional evaluation component that estimates test sensitivity beyond structural coverage.

2.2 Program Representation and Lightweight Code Analysis

Although the framework does not implement heavyweight program analysis (e.g., symbolic execution), it still relies on lightweight static structure to ensure generated tests are executable and well-formed. The Python AST serves as a convenient intermediate representation: by parsing candidate test files into an AST, the system can validate syntax, identify `test_` functions, and enforce simple constraints such as test atomicity (one assertion per test) to improve diagnostics. These checks are essential because LLM outputs may be truncated, include extraneous prose, or contain placeholders; without validation, execution would be dominated by avoidable failures.

2.3 Large Language Models for Code and Domain Adaptation

LLMs can be viewed as probabilistic code synthesizers trained to predict tokens conditioned on context. Few-shot prompting demonstrated that large transformer models can perform new tasks from natural-language instructions [6], and work on code-specialised models shows strong code generation capability [7]. These properties make LLMs attractive for test generation, since tests are code artefacts with strong regularities (imports, fixtures, assertions).

Off-the-shelf LLMs face practical constraints: multiple valid test suites can exist for the same module, prompt compliance can be brittle, and deployment is resource-bound. This project therefore favours a smaller open-source code model adapted to the task and executed locally. Parameter-efficient fine-tuning enables adaptation by training a small number of additional parameters while freezing the base model [13], and quantization compresses weights to make CPU inference feasible. A locally runnable model reduces cloud dependency and source-code exposure, but increases the importance of careful prompting and validation.

2.4 Prompt Engineering and Iterative Refinement

Prompt engineering conditions an LLM on both the code under test and the constraints required by the execution environment. In this project, prompts explicitly request `pytest` test files, provide the sandbox import path, and enforce a concise, machine-checkable output format. These constraints operationalise a simple principle: generated artefacts must be executable before they can be evaluated for coverage or oracle quality.

The framework also applies prompt- and parser-level measures to keep generation tractable under limited context windows. Before inclusion in prompts, source files are simplified by

removing non-essential tokens (notably line comments) while retaining docstrings, which often encode preconditions, return semantics, and usage examples. The sandboxing mechanism further constrains the generation task by fixing an explicit import target for the code under test (a unique module name inside an isolated package), avoiding ambiguity about relative imports and reducing the likelihood of name collisions when tests are executed. After generation, strict post-processing and validation are applied to enforce the requested output format (raw test file code only) and to filter out structurally unhelpful patterns such as placeholder tests. This combination of input normalisation, explicit execution context, and syntactic validation is a practical response to the fact that LLMs can produce superficially plausible but non-runnable code.

Execution feedback closes the loop between generation and validation. When a generated test fails to import, raises an exception, or violates a structural constraint, the pipeline captures the failure trace and uses it as context for refinement. Recent evaluations of ChatGPT-style test generation suggest that iterative correction can improve executability, while oracle errors remain common [8, 14]. The framework’s refinement loop embodies this idea by repeatedly repairing tests until they execute or a refinement budget is exhausted.

2.5 Evaluation Signals: Executability, Coverage, and Effectiveness

Because fully determining correctness is often infeasible, practical evaluation combines multiple signals. The first is executability: whether the produced test file runs under `pytest` without errors. The second is structural adequacy, measured here using statement coverage, which should be interpreted as exploration rather than correctness [5]. The third, optional signal is mutation score, which estimates sensitivity to small behavioural perturbations [10, 11]. Together, these signals reflect the design intent of the framework: to produce small, executable unit tests that explore code paths and offer a useful starting point for developer review in the presence of the oracle problem [4].

3 LITERATURE REVIEW

This chapter reviews research that underpins and contextualises the proposed LLM-assisted software testing framework. The framework developed in this thesis targets repository-scale unit test synthesis for Python by generating `pytest` tests with a local large language model (LLM), executing them with `Coverage.py` instrumentation inside a per-file sandbox, and iteratively refining failing tests using execution feedback. The literature is organised around four intersecting themes: (i) automated unit test generation (symbolic, concolic, search-based, and random/property-based), (ii) adequacy metrics and the test-oracle problem, (iii) mutation testing as a complementary adequacy signal, and (iv) LLM-based code and test generation systems and their empirical evaluation. The discussion emphasises how prior work addresses coverage, scalability, and reliability, and it relates each thread to design choices and observed limitations of the current framework.

3.1 Automated Unit Test Generation: Core Paradigms and Practical Trade-offs

3.1.1 Symbolic execution as a foundation for systematic exploration

King introduced symbolic execution to address the fundamental difficulty of constructing inputs that drive programs down specific control-flow paths [15]. In symbolic execution, inputs are treated as symbols rather than concrete values; execution accumulates path constraints that characterise the conditions under which each path is feasible. A constraint solver can then be used to produce concrete inputs satisfying those constraints, enabling highly targeted testing of branches that are rare under random input selection. The conceptual strength of the approach is that test generation is driven by semantic reasoning over program paths. However, classic limitations appear immediately in realistic software: the number of paths grows rapidly (path explosion), constraints can be hard or undecidable for complex operations, and interacting with file systems, networks, and external libraries requires modelling choices that can dominate engineering effort.

The present thesis adopts the goal that originally motivated symbolic execution (reach more behaviours with less manual input design), but it uses a different mechanism. Rather than solving constraints, the framework prompts an LLM to propose concrete unit tests and validates them via execution. The consequence is a shift in what is “systematic”: symbolic execution is systematic in path coverage by construction, while an LLM-driven pipeline is systematic primarily in orchestration (repeatable generation, sandboxed execution, and structured artefact logging). The results later in this thesis show that this can yield high statement coverage with

low generation cost on small modules, but without the coverage guarantees associated with solver-driven exploration.

3.1.2 Concolic testing: pragmatic symbolic reasoning grounded in concrete runs

Concolic testing combines concrete execution with symbolic constraint collection to mitigate the practical barriers of purely symbolic exploration. DART proposed a directed approach in which each run is executed concretely while collecting symbolic constraints along the observed path; subsequent tests are generated by negating selected constraints to force alternative branches [16]. The central problem addressed is the inefficiency of undirected random testing for reaching deep branches: by systematically manipulating constraints, concolic testing can traverse branch conditions that are unlikely to be satisfied by chance. A key finding across this line of work is that concolic methods can be effective without requiring full symbolic modelling of all operations, but they still depend on solver capability and on heuristics for selecting which constraints to negate.

CUTE adapted concolic testing to unit testing contexts, demonstrating that function-level exploration can be practical when the environment is controlled and the program interfaces are amenable to symbolic reasoning [17]. In the .NET ecosystem, Pex operationalised similar ideas in a developer-facing tool, emphasising usability and integration with existing workflows [18]. Methodologically, these systems are significant because they treat test generation as both an analysis problem (constraints, symbolic states) and a tooling problem (integration, usability, execution stability). Their limitations are similarly dual: scaling to complex software can be constrained by solver complexity and by the engineering costs of modelling environments and dependencies.

Relative to these systems, the framework of this thesis seeks a different kind of portability. It deliberately avoids dependence on constraint solvers and deep program analysis, instead relying on the LLM as a proposal mechanism and `pytest` as the execution verifier. The trade-off is that this pipeline does not guarantee systematic exploration of uncovered branches. However, it gains practical advantages in language portability (standard Python), deployment simplicity (no specialised analysis tooling), and offline execution using a local model.

3.1.3 Random and property-based testing: low-infrastructure generation with strong oracles

Random testing remains influential because it reduces the complexity of input generation and can still discover failures effectively under certain conditions. Duran and Ntafos empirically studied random testing and highlighted that its effectiveness depends on how frequently failure-inducing inputs occur in the input domain [9]. When bugs are triggered by broad

regions of the input space, random testing can find failures quickly; when bugs are narrow corner cases, random testing may require prohibitive sampling.

Property-based testing can be understood as a structured evolution of random testing in which a domain-specific oracle is expressed as a property. QuickCheck demonstrated that expressing behaviour as properties, then randomly generating inputs to challenge those properties, can uncover subtle defects at scale [19]. The methodology is notable for combining a strong oracle (properties) with automated exploration (random generation plus shrinking). Its practical limitation is that it presumes the availability of properties; when intent is undocumented or implicit, writing properties can be as hard as writing tests.

The framework in this thesis shares the low-barrier appeal of random/property-based approaches: it aims to generate immediately executable tests that can be integrated into standard `pytest` workflows. However, unlike QuickCheck, it does not require explicit developer-written properties. Instead, it depends on the LLM to infer plausible assertions from code and docstrings. This effectively moves oracle construction from explicit specification (properties) into probabilistic inference, which increases the risk of oracle errors when the intended semantics are ambiguous.

3.1.4 Feedback-directed random testing and regression-style assertions

Feedback-directed random testing (FDRT) addresses the challenge of assertion creation by using observed program behaviour as a surrogate oracle. Randoop automatically generates sequences of calls and then synthesises regression assertions from outcomes observed during execution [1]. The work targets the problem that many real systems have incomplete specifications: regression-style tests still provide value by detecting behavioural changes across versions. A key strength is scalability and low specification burden. A key limitation is brittleness: if observed behaviour is wrong, or if correct behaviour changes legitimately, regression assertions can become noisy or misleading.

This thesis’ framework encounters a related phenomenon in a different form. LLM-generated assertions can also be brittle, but for a different reason: they may encode a plausible but incorrect interpretation of intent rather than a snapshot of observed behaviour. Execution feedback can detect disagreement, but without an external oracle it cannot determine which side is correct. This motivates the framework’s design choice to preserve artefacts and present failure excerpts as part of the workflow, supporting human review when semantic disputes arise.

3.1.5 Search-based test generation and assertion synthesis

Search-based software testing (SBST) frames test generation as an optimisation problem. McMinn’s survey synthesised how fitness functions such as branch distance and approach level can guide metaheuristic search toward coverage goals [2]. This approach can be effective when constraint solving is hard, but it requires careful fitness design and can be computationally expensive for slow-running systems or complex input structures.

EvoSuite is a widely cited example of search-based unit test generation that targets high coverage and synthesises assertions to turn executions into regression tests [3]. Its key finding is that automatic suites can achieve high structural coverage and provide a baseline test suite where none exists, but limitations include unstable assertions, flakiness, and the risk that generated checks overfit to incidental behaviours. These limitations are conceptually related to the oracle issues observed in LLM-generated tests: reaching code is feasible, but encoding correct expected behaviour is substantially harder.

Compared to SBST, the pipeline in this thesis can be interpreted as a “proposal + filter” strategy rather than a direct optimisation loop. The LLM proposes tests, execution filters out syntactically invalid or crashing tests, and refinement attempts repair. A natural research gap emerges at this boundary: integrating coverage feedback as an explicit guidance signal to steer LLM prompts toward uncovered branches could combine SBST’s coverage focus with LLMs’ ability to rapidly scaffold tests.

3.2 Adequacy, Coverage, and the Test Oracle Problem

3.2.1 Coverage as a partial and sometimes misleading indicator

Coverage is widely used because it is easy to compute and provides a simple measure of whether code has been executed. However, its relationship to fault detection is nuanced. Inozemtseva and Holmes empirically showed that coverage is not strongly correlated with test suite effectiveness in general, cautioning against interpreting high coverage as a direct proxy for quality [5]. Their work addresses the problem of evaluation metrics: if coverage is used as the primary target, developers may obtain suites that execute statements but do not detect faults reliably.

This perspective aligns with this thesis’ evaluation design. The framework reports statement coverage as evidence that generated tests exercise code paths, but it does not claim that coverage alone implies semantic correctness. Empirically, a test suite may reach full statement coverage while failing due to incorrect assertions; conversely, a suite may pass while still missing critical behaviours. Therefore, coverage is treated as a descriptive metric that complements executability and qualitative failure analysis.

3.2.2 The oracle problem as the dominant barrier to full automation

Barr *et al.* surveyed the oracle problem and clarified why automated testing frequently struggles to determine correctness when ground truth is absent [4]. The survey categorises oracle strategies and highlights trade-offs between precision, cost, and applicability. A central conclusion is that automated test generation is often limited not by the ability to generate inputs or execute code, but by the ability to determine whether outcomes are correct.

In LLM-assisted unit testing, this bottleneck becomes especially salient. LLMs can generate plausible assertions and edge cases, but plausibility is not correctness. When failures occur, execution feedback indicates mismatch, yet it cannot automatically resolve whether the code or the generated oracle is wrong. The refinement stage in this thesis improves mechanical reliability (imports, fixtures, syntax validity, and API mismatches), but it is less effective when the failure is conceptual (e.g., domain rules such as leap-year edge cases or behaviour implied by ambiguous documentation). Consequently, the oracle problem is the primary source of residual risk in the framework.

3.3 Mutation Testing and Stronger Adequacy Signals

3.3.1 Mutation testing as a complement to coverage

Mutation testing assesses adequacy by seeding faults and measuring whether tests detect them. Jia and Harman surveyed mutation testing, showing that mutation score can be a stronger proxy for fault detection than structural coverage alone [11]. Their analysis emphasises both the promise of mutation testing and its practical challenges: equivalent mutants, the cost of executing many mutants, and the need to interpret surviving mutants carefully. Classical mutation analysis traces back to early work such as DeMillo *et al.* [10], which motivated mutation operators as a principled way to evaluate test sensitivity.

The current project’s codebase includes optional mutation-testing utilities, but the main evaluation in this thesis focuses on executability and coverage due to cost constraints and the emphasis on pipeline automation. The literature suggests a clear extension: integrating mutation testing as an optional evaluation stage could quantify whether LLM-generated tests detect behavioural deviations rather than merely execute code. This becomes increasingly important as LLM-based generation can achieve high coverage quickly; mutation score would help discriminate between tests that are “covering” and tests that are “checking”.

3.4 LLMs for Code and Test Generation

3.4.1 LLMs as code generators and the role of prompting

Large language models have changed practical software engineering by enabling code generation and editing through natural-language prompting. Brown *et al.* demonstrated that sufficiently large autoregressive models exhibit strong few-shot capabilities across tasks, motivating prompting as a general interaction paradigm [6]. Code-specific pretraining approaches, such as CodeBERT [20] and CodeT5 [21], further illustrate that training on code corpora can improve model competence in code understanding and generation.

For testing, these models are attractive because they can rapidly produce scaffolding: imports, fixtures, and plausible call patterns. However, generation is probabilistic and may embed incorrect assumptions about APIs or behaviour. This thesis’ framework therefore pairs generation with execution-based validation and lightweight normalisation steps, reflecting a broader trend in LLM-based programming: models are most reliable when coupled with external verifiers.

3.4.2 Execution-based evaluation and its mirror image in test generation

Chen *et al.* evaluated language models trained on code and introduced execution-based evaluation (HumanEval), highlighting that repeated sampling and selection improves success rates [7]. The methodological point is that functional correctness must ultimately be assessed by running code against tests.

This thesis adopts a related philosophy in the opposite direction. Rather than generating programs and validating them with tests, it generates tests and validates them by running them against the program. The limitation is symmetrical: running generated tests can validate that they execute and that assertions hold for observed behaviour, but it cannot guarantee that the assertions encode the intended specification. Therefore, the oracle problem remains the dominant limitation even under rigorous execution feedback.

3.5 LLM-Based Unit Test Generation: Empirical Findings and System-Level Lessons

3.5.1 Evaluating ChatGPT-style generation and improving reliability

Yuan *et al.* studied whether LLMs can replace or reduce manual effort in unit test creation, evaluating and improving ChatGPT for unit test generation [8]. The problem addressed is practical: developers want test suites that are executable, relevant, and reasonably correct with minimal prompting overhead. The methodology typically combines prompt-based

generation with evaluation on real code, measuring metrics such as compilation/execution success, coverage, and qualitative correctness. A key empirical finding across such studies is that LLMs often produce syntactically valid test scaffolding and can boost coverage, but correctness is uneven: missing edge cases, incorrect assertions, and unrealistic mocking are recurrent limitations.

This is closely aligned with the pipeline observed in this thesis. The framework’s sandboxing and forced headers primarily target mechanical reliability. Where the literature highlights correctness and oracle issues, the framework’s refinement loop and artefact logging provide a practical response: tests are not merely generated, but executed, repaired, and preserved so that failures can be inspected and corrected.

3.5.2 Comparing LLM generation with SBST baselines

Tang *et al.* compared ChatGPT-style approaches with search-based unit test generation (SBST) [14]. The underlying question is where LLMs sit in the automated testing design space: do they outperform or complement classical coverage-driven search? Typical findings indicate that LLMs may generate plausible, human-readable tests quickly and sometimes reach good coverage with low configuration, while SBST tools provide more systematic exploration but may produce less readable tests or require more tooling setup. Limitations for LLM approaches frequently include assertion correctness and overconfidence in inferred behaviour.

For the current framework, this comparison motivates two interpretations. First, LLMs are effective proposal mechanisms under tight time budgets. Second, SBST remains an important reference point when systematic coverage targeting is required, suggesting a hybrid future direction where coverage feedback informs prompts or where LLMs propose candidates that are then diversified by search.

3.5.3 Evaluation methodology for LLM unit test generation

Yang *et al.* discussed evaluation of LLMs in unit test generation [22]. A recurring methodological issue is that simple metrics such as compilation success or statement coverage can overstate success if oracles are wrong or if tests are shallow. Robust evaluation therefore often combines executability, coverage, mutation score, and manual inspection of assertion intent. This work reinforces the evaluation design of this thesis: coverage is informative but insufficient, and failure analysis should focus on semantic mismatches.

3.5.4 Mutation-guided and signal-guided LLM test generation

Recent research explores using stronger feedback signals to improve the semantic adequacy of LLM-generated tests. For example, mutation-guided unit test generation uses mutation

outcomes to drive the generation or refinement process, attempting to strengthen tests beyond mere coverage [23]. Other work proposes steering mechanisms based on compilable prefixes or branch signals [24], and path-sensitive strategies that seek to connect LLM prompting with coverage objectives [25]. These directions are directly relevant to the present framework, which already incorporates a refinement loop and coverage instrumentation. The literature suggests that adding richer feedback (mutation score, branch coverage, uncovered-line targeting) could transform refinement from “make it pass” into “make it discriminative”.

3.6 Synthesis and Research Gaps Motivating This Thesis

Across the reviewed literature, three gaps are particularly relevant to the framework proposed in this thesis. First, many highly effective automated test generation techniques (symbolic/concolic and search-based) require substantial analysis infrastructure [15, 16, 2]. LLM-based methods offer a pragmatic alternative for quickly producing executable tests with low tooling requirements, but they shift the core difficulty toward oracle correctness and evaluation. Second, coverage-oriented metrics remain attractive but can be misleading as quality indicators [5]; combining coverage with executability, refinement effort, and stronger adequacy metrics (e.g., mutation score) is therefore necessary for credible assessment [11]. Third, empirical studies of LLM-based unit test generation show that scaling requires end-to-end orchestration and reliable execution environments [8, 22]. This aligns with the implementation emphasis of this thesis: sandboxing, logging, artefact capture, and interactive control decisions to maintain throughput.

The framework introduced in this thesis is positioned as a practical, offline-capable pipeline that integrates LLM generation with execution feedback and coverage measurement for Python repositories. Its novelty is not a claim that LLMs eliminate established testing theory, but that they can be embedded into a reproducible workflow whose outcomes are measurable and extensible. The experimental results later in this thesis demonstrate the resulting trade-offs: high coverage and high executability for small modules, alongside persistent oracle limitations that define the boundary of fully automated unit testing.

3.7 Comparison of Related Work

Table 3 provides a structured comparison of more than 35 research works spanning automated test generation, adequacy and oracle research, mutation testing, and LLM-based unit test generation. The table is set in an `xltabular` environment so it fits within page margins and can comfortably span multiple pages.

Table 3: Comparison of related work and its relevance to the proposed framework.

Reference	Year	Category	Main idea / method	Relevance to this thesis
[15]	1976	Symbolic exec.	Symbolic inputs + path constraints + solver-driven input construction; exposes path explosion.	Motivates systematic coverage goals; contrasts with LLM proposal strategy.
[10]	1978	Mutation	Early mutation analysis framing; mutation operators for test evaluation.	Historical basis for using mutation as adequacy measure beyond coverage.
[9]	1984	Random testing	Empirical analysis of random input selection as a lightweight baseline.	Baseline for low-infrastructure generation; LLM tests are a guided alternative.
[26]	1990	Fuzzing (black-box)	Random input “fuzz” experiments revealing reliability issues in real utilities.	Motivates random exploration as a complementary discovery technique.
[19]	2000	Property- based	Random generation guided by properties; shrinking failing inputs.	Strong-oracle alternative to inferred assertions; highlights property bottleneck.
[27]	2001	Regression	Test prioritisation for earlier fault revelation under budget constraints.	Supports fast-feedback design and budgeted refinement execution loops.
[28]	2004	Random variants	Adaptive random testing increases diversity by spreading tests across the input domain.	Suggests alternative input diversification beyond naive random selection.

Reference	Year	Category	Main idea / method	Relevance to this thesis
[2]	2004	SBST survey	Optimisation-based test generation using fitness functions (e.g., branch distance).	Positions coverage feedback as future hybrid guidance for LLM generation.
[16]	2005	Concolic	Concrete runs + symbolic constraints; directed exploration by constraint negation.	Reference point for systematic path exploration; contrasts with LLM pipeline.
[17]	2005	Concolic unit	Concolic engine specialised for unit testing; practical unit-level exploration.	Highlights analysis-heavy alternative avoided by the proposed framework.
[29]	2006	Symbolic exec.	Automatic generation of failure-inducing inputs (EXE).	Reinforces execution infrastructure and modelling as key practical factors.
[1]	2007	FDRT	Feedback-directed random generation; regression assertions derived from observations.	Closest analogue for oracle brittleness; LLM assertions can be similarly brittle.
[30]	2008	Symbolic exec.	High-coverage test generation for complex programs via symbolic execution and modelling.	Supports systems perspective: robust execution and models matter at scale.
[31]	2008	Whitebox fuzz	Whitebox fuzzing (SAGE) combining symbolic reasoning with large-scale testing.	Motivates hybrid “generate + execute” pipelines and engineering trade-offs.

Reference	Year	Category	Main idea / method	Relevance to this thesis
[18]	2008	White-box unit	Pex: dynamic symbolic execution integrated into unit test workflows.	Contrast: solver-driven coverage targeting vs. LLM-driven scaffolding.
[3]	2011	Auto unit suites	Search-based unit test generation + assertion synthesis; coverage-driven.	Baseline for auto suites; informs assertion fragility discussion.
[11]	2011	Mutation survey	Mutation score as adequacy metric; challenges include cost and equivalent mutants.	Motivates extending evaluation beyond coverage to mutant killing.
[32]	2011	Differential testing	Random program generation to find compiler bugs (differential strategy).	Illustrates oracle alternatives (differential consistency) relevant to future work.
[5]	2014	Adequacy	Empirical study: coverage is weakly correlated with fault detection.	Justifies cautious interpretation of coverage metrics reported in this thesis.
[12]	2014	Mutation validity	Study of mutants vs. real faults for evaluating test suites.	Guides interpretation of mutation score if integrated into evaluation.
[4]	2015	Oracle survey	Taxonomy and analysis of oracle strategies; correctness bottleneck persists.	Explains persistent semantic failures (assertion mismatches) observed here.
[33]	2016	Metamorphic	Survey of metamorphic testing for domains without explicit oracles.	Suggests future oracle strategies for ambiguous behaviours.

Reference	Year	Category	Main idea / method	Relevance to this thesis
[34]	2021	Fuzzing survey	Survey of fuzzing engineering and evaluation; oracle remains hard.	Alternative exploration mechanism complementing unit tests and coverage.
[6]	2020	LLM foundations	Scaling and prompting yield strong few-shot behaviour (GPT-3).	Supports prompting as a control surface for generation/refinement stages.
[20]	2020	Code pretrain	Joint NL+code pretraining for code understanding/generation (CodeBERT).	Motivates code-aware modelling for test generation from source context.
[21]	2021	Code pretrain	Unified encoder-decoder pretraining for code tasks (CodeT5).	Background for code-specialised modelling applicable to test generation.
[7]	2021	Execution eval.	HumanEval-style execution-based correctness evaluation for code LMs.	Mirrors this thesis’ “execute to validate” philosophy.
[35]	2022	Code generation	Large-scale code generation models (CodeGen).	Explains why scaffolding is often easy while semantic correctness remains hard.
[36]	2022	Code editing	Code infilling/editing model (InCoder) useful for repair workflows.	Aligns with refinement/repair framing for failing generated tests.
[37]	2022	Program synthesis	Sample-and-select code generation at high difficulty (AlphaCode).	Motivates multi-sample test generation with execute-filter selection.

Reference	Year	Category	Main idea / method	Relevance to this thesis
[38]	2023	Open code LMs	Open code LMs enabling local/offline inference (StarCoder).	Aligns with offline local model usage in this thesis.
[8]	2023	LLM test gen.	Empirical evaluation and improvement of ChatGPT for unit test generation.	Closest empirical analogue for executability and assertion correctness challenges.
[14]	2023	LLM vs SBST	Comparative evaluation of ChatGPT-style test generation vs SBST baselines.	Supports positioning: fast LLM proposals vs systematic search-based exploration.
[39]	2023	Tool comparison	Comparative analysis of generative-AI test autogeneration tools.	Reinforces evaluation needs: metrics must include correctness, not only coverage.
[22]	2024	Evaluation study	Methodology for evaluating LLMs in unit test generation (coverage/executability/correctness).	Motivates multi-metric evaluation and careful interpretation of coverage.
[13]	2024	Fine-tuning	Parameter-efficient fine-tuning for unit test generation; empirical study.	Relevant to local-model adaptation (domain fine-tuning) in future work.
[40]	2024	Readability	Improving understandability of generated unit tests with LLMs.	Connects to practical adoption: readable tests ease human review of oracles.

Reference	Year	Category	Main idea / method	Relevance to this thesis
[23]	2025	Mutation-guided	Mutation-guided LLM unit test generation to strengthen adequacy.	Directly supports future integration of mutation feedback into refinement.
[41]	2025	Test design	LLM unit tests guided by equivalence partitions and boundary values.	Suggests prompt-level guidance aligning with classical test design heuristics.
[24]	2025	Signal-guided	Compilable-prefix and branch-signal steering for LLM unit test generation.	Motivates adding coverage/branch signals to steer generation toward gaps.
[25]	2025	Path-sensitive	Path-sensitive unit test generation strategies for LLMs.	Aligns with future hybrid approach: LLM + path/coverage targeting.

4 PROPOSED FRAMEWORK

This chapter presents the design and architecture of the proposed framework for automated software testing driven by a quantized large language model. The framework addresses the end-to-end workflow of generating, validating, executing, and iteratively refining unit tests for Python source code repositories, requiring minimal human intervention while producing high-quality, executable test suites with measured code coverage. The overall system architecture is illustrated in Figure 1, and the detailed stage-by-stage pipeline is shown in Figure 2.

4.1 Purpose and Motivation

Software testing remains one of the most labour-intensive activities in modern software engineering. Manual test authoring demands deep understanding of the code under test, is prone to human error, and scales poorly as codebases grow. Industry surveys consistently report that developers spend between 30% and 50% of their time on testing-related tasks, yet many projects still suffer from inadequate test coverage. These factors have motivated a growing body of research into automated test generation techniques that can reduce the burden on developers while maintaining or improving the quality of test suites.

Traditional automated testing tools such as Randoop, EvoSuite, and Pynguin rely on search-based or random strategies that, although effective for achieving structural coverage, frequently produce tests that lack semantic meaning and are difficult for developers to understand or maintain. The emergence of large language models capable of generating syntactically and semantically coherent code has opened a new avenue for test generation that more closely resembles developer-written tests. Recent studies have demonstrated the potential of LLM-based approaches: TestForge achieved an 84.3% pass rate at a cost of only \$0.63 per file through iterative refinement, while LlamaRestTest showed that fine-tuned and quantized smaller models can outperform larger GPT-powered tools in REST API testing through 2-bit, 4-bit, and 8-bit quantization.

However, deploying state-of-the-art LLMs such as GPT-4 or Claude for continuous test generation introduces significant challenges. Cloud-based API calls incur recurring financial costs, raise data privacy concerns when proprietary source code must be transmitted to external servers, and introduce latency that hinders integration into rapid development cycles. Furthermore, large commercial models with hundreds of billions of parameters require substantial computational infrastructure that is impractical for most development teams and individual researchers.

The proposed framework addresses these limitations by employing a domain-specific, quantized open-source model—Qwen2.5-Coder-1.5B-Instruct, fine-tuned with QLoRA and exported to GGUF Q4_K_M format—that runs entirely on local hardware using CPU inference via `llama-cpp-python`. This design choice yields a model of approximately 941 megabytes that can execute on commodity hardware without a GPU, eliminating cloud dependency, preserving source code confidentiality, and enabling offline operation. The framework wraps this quantized model within a multi-stage pipeline that automates the entire lifecycle of test generation, from source code discovery through iterative refinement based on execution feedback.

4.2 High-Level System Architecture

The framework is organised as a modular, layered system comprising five principal stages: input and code analysis, prompt engineering, LLM-based test generation, validation and filtering, and test execution with iterative feedback. Figure 1 presents the high-level view of this architecture, showing the flow from source code files as input through each processing layer to the final outputs of an executable test suite, coverage reports, and quality metrics.

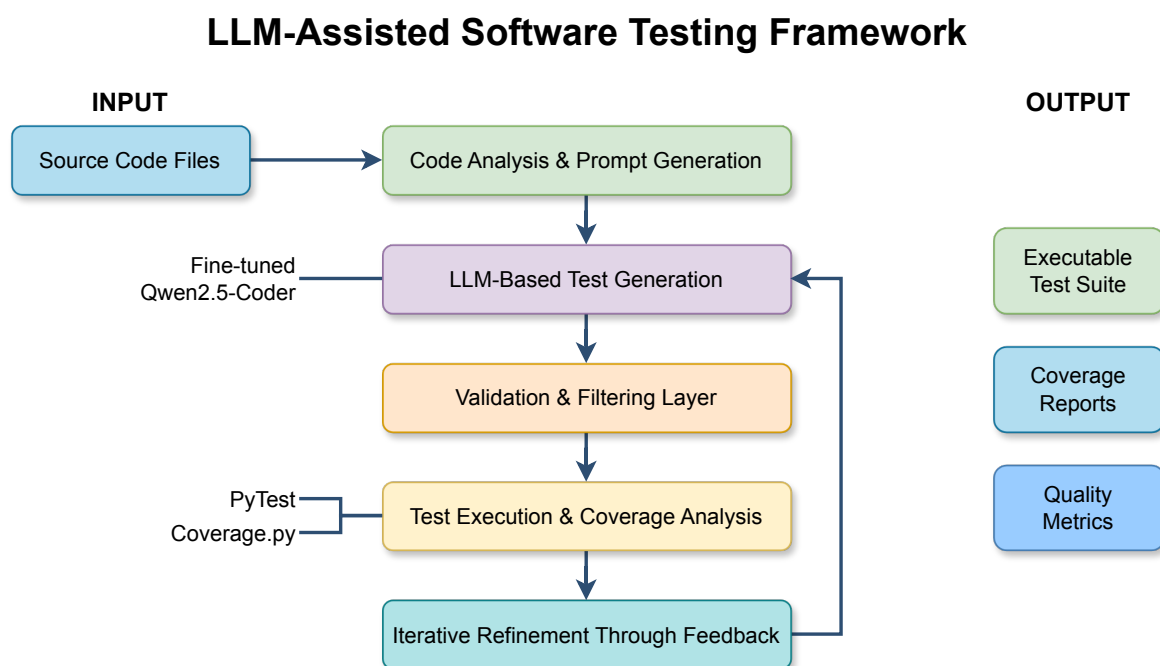


Figure 1: System architecture of the LLM-assisted software testing framework.

At the highest level, the system accepts a Python source code repository as input through either a command-line interface or a Flask-based web application. The repository passes through the code analysis and prompt generation stage, where source files are discovered, filtered, and prepared for processing. The prepared code context is then fed into the LLM-based test generation stage, which invokes the fine-tuned Qwen2.5-Coder model to produce raw PyTest

test cases. These generated tests enter a validation and filtering layer that performs syntactic checking, import validation, semantic analysis, duplicate detection, and placeholder removal. Valid tests proceed to the test execution and coverage analysis stage, where PyTest runs the tests and Coverage.py measures code coverage. If tests fail, the framework enters an iterative refinement loop through which failure feedback is incorporated into a new prompt, and the LLM generates refined tests. This cycle repeats for a configurable number of attempts before the system either marks the file as complete or escalates for user decision.

4.3 Framework Pipeline: Stage-by-Stage Design

The detailed five-stage pipeline is depicted in Figure 2. Each stage is implemented as a distinct module within the codebase, promoting separation of concerns and facilitating independent testing and extension of individual components.

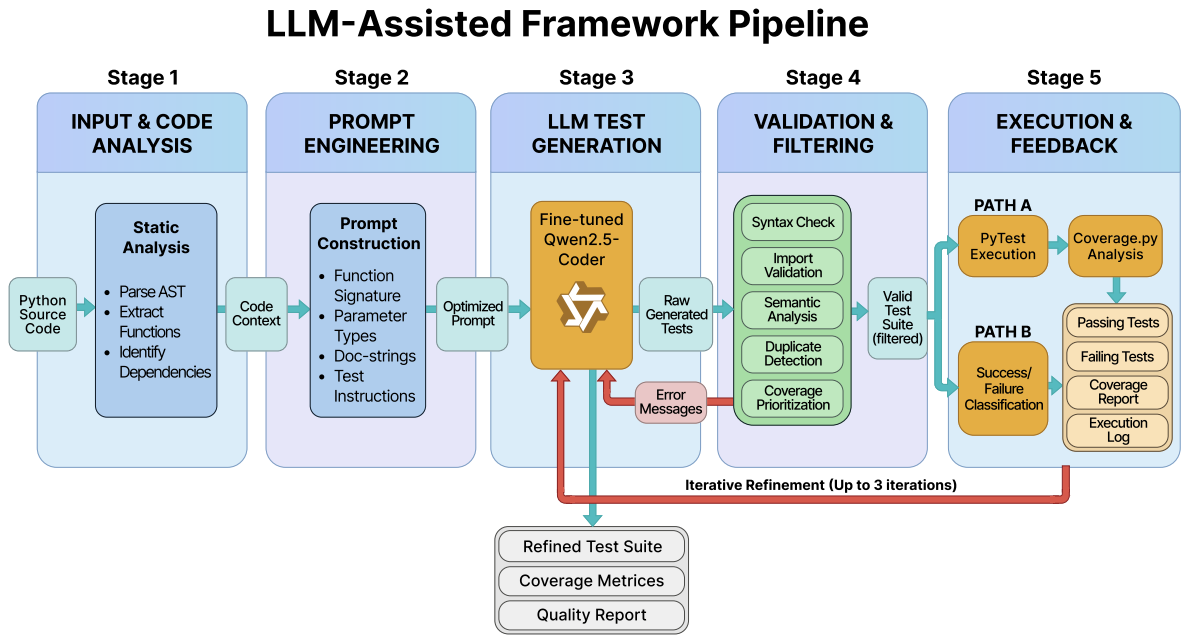


Figure 2: Five-stage pipeline for LLM-assisted unit test generation and refinement.

4.3.1 Stage 1: Input and Code Analysis

The first stage is responsible for ingesting a Python project repository and extracting the source files that are suitable candidates for test generation. The `pipeline/discovery.py` module implements recursive file discovery, traversing the directory tree while excluding common non-source directories such as `.git`, `__pycache__`, virtual environments, build artefacts, and previously generated experiments. A set of heuristic filters further eliminates files that are unlikely to benefit from unit testing, including empty `__init__.py` files, Django migration scripts, `setup.py` configuration files, and `conftest.py` fixtures.

Each discovered source file is then copied into an isolated sandbox environment within the run directory. The orchestrator creates a dedicated `eval_sandbox` package, assigning each source file a unique module name of the form `mod_NNNN.py`. This sandboxing strategy serves multiple purposes: it prevents import conflicts between the original project modules and the generated test files, it provides a controlled namespace for test execution, and it ensures that the original repository files are never modified during the testing process. The corresponding import path `from eval_sandbox.mod_NNNN import *` is constructed at this stage and passed forward to subsequent pipeline stages.

Before the code is incorporated into a prompt, the analysis module applies a comment-stripping transformation using Python's `tokenize` module. This operation removes hash-style comments while preserving docstrings, reducing token consumption without losing the structural and documentation cues that the LLM relies upon for understanding function behaviour.

4.3.2 Stage 2: Prompt Engineering

The prompt engineering stage constructs the instruction that will be sent to the quantized LLM. The framework employs a structured prompt template defined in `llm/prompt_templates.py` that follows the instruction-response format compatible with the ChatML chat template used by Qwen2.5-Coder. The generation prompt comprises several carefully designed components:

- A system-level instruction establishing the model's role as an expert software testing assistant
- An explicit constraint to produce only PyTest-format test cases, excluding unittest or other formats
- A rule enforcing that each test function contains exactly one assertion, promoting test atomicity and clearer failure diagnostics
- A prohibition against placeholder tests that merely `assert True`
- A token budget constraint of 1,200 tokens to prevent truncated output
- The sandbox import path for correct module referencing
- The preprocessed source code under test

The decision to enforce single-assertion test functions reflects a deliberate design philosophy. When each test function tests exactly one behaviour, a failing test immediately identifies which aspect of the code is broken. This granularity also produces more meaningful coverage metrics, since each test contributes an independently measurable unit of coverage. The token budget constraint of 1,200 tokens was determined empirically during development: it provides sufficient space for generating multiple meaningful test functions for typical Python modules

while preventing the model from entering degenerate generation patterns that arise when output approaches the model’s context window limit.

For the refinement prompt, a separate template extends the generation prompt with three additional contextual elements: the existing test code that failed, the pytest error output captured from the previous execution attempt, and explicit instructions to fix faulty imports, remove duplicates, improve correctness and edge case coverage, and convert any unittest-style tests to PyTest format. This error-conditioned prompting enables the model to diagnose specific failure modes and apply targeted corrections rather than regenerating tests from scratch.

4.3.3 Stage 3: LLM-Based Test Generation

The core of the framework is the test generation stage, implemented in `llm/test_generator.py`. This module receives the constructed prompt and invokes the quantized Qwen2.5-Coder model through the `llm/LLM_models.py` wrapper, which loads the GGUF Q4_K_M model file using `llama-cpp-python` with a context window of 3,072 tokens, six CPU threads, and a batch size of 256.

The model is invoked using the `create_chat_completion` API with deterministic decoding parameters: a temperature of 0.0 for greedy generation during the standard pipeline (or 0.2 in simple mode), a repetition penalty of 1.05 to discourage degenerate repetition, and a maximum output of 1,200 tokens. Greedy decoding was selected for the production pipeline to ensure reproducibility across runs—given the same source code input, the framework produces identical test suites, which is a desirable property for regression testing and scientific evaluation.

The raw output from the model undergoes a multi-step post-processing pipeline within the same module. First, the `extract_code_block_or_raw` function searches for fenced code blocks (delimited by triple backticks) in the model’s response and extracts the longest such block, handling cases where the model wraps its output in markdown formatting. If no code blocks are found, the entire response is treated as raw code. Next, the `force_header` function ensures that the generated code includes the required `import pytest` statement and the sandbox import path. This function preserves any additional imports that the model generated while inserting missing ones at the correct position after any shebang lines, encoding declarations, or module docstrings.

A critical post-processing step is `salvage_truncated_output`, which handles a common failure mode of quantized small models: incomplete code generation. When the model reaches its token limit mid-function, the resulting code contains syntax errors. The salvage function iteratively removes lines from the end of the output until the remaining code parses as valid

Python and contains at least one test function. This heuristic recovers usable test code in many cases that would otherwise require a full regeneration attempt.

The `enforce_one_assert_per_test` function uses Python’s Abstract Syntax Tree (AST) module to identify test functions containing multiple assertions. When such functions are found, they are split into separate test functions, each retaining the setup code (all non-assert statements preceding the assertion) and a single assert statement. The split functions are renamed with numeric suffixes (e.g., `test_add_01`, `test_add_02`) to maintain uniqueness. Finally, `_drop_placeholder_tests` uses AST analysis to remove any test functions whose body consists solely of `assert True`, which the model occasionally produces as filler when it runs out of meaningful test ideas.

4.3.4 Stage 4: Validation and Filtering

The validation stage applies a multi-layered quality gate to the post-processed test code before it is executed. Rather than implementing validation as a single monolithic check, the framework distributes validation logic across the post-processing functions described above and the orchestrator’s execution loop, creating a defence-in-depth approach.

Syntactic validation is performed through `ast.parse()`, which verifies that the generated code is valid Python. Tests that fail to parse after the salvage attempt are rejected. Import validation ensures that the generated tests correctly reference the sandbox module path and include the `pytest` import. Semantic validation is achieved through the placeholder detection mechanism, which removes tests that do not contain meaningful assertions. Duplicate detection, implemented in the refinement pipeline through `deduplicate_tests_by_ast`, uses AST structural comparison to identify and eliminate test functions that are semantically identical despite differing in whitespace or variable naming. The function normalises test function names before comparison, ensuring that only structurally distinct test functions survive.

The validation pipeline applies these checks in a specific order: header enforcement, truncation salvage, assertion splitting, and placeholder removal. This ordering is significant because earlier transformations may produce artefacts that later transformations clean up. For instance, the assertion-splitting step may create functions that are structurally identical to existing functions, which the deduplication step then removes during refinement.

4.3.5 Stage 5: Test Execution, Coverage Analysis, and Iterative Feedback

The final stage of the pipeline encompasses test execution, coverage measurement, and the iterative refinement loop. The `pipeline/coverage.py` module manages test execution by invoking `coverage run -m pytest` as a subprocess, which simultaneously runs the

generated tests and measures which lines of the source code under test are exercised. The execution uses `-maxfail=1` to fail fast on the first error, providing rapid feedback for the refinement loop, and `-q` for quiet output that minimises noise in the parsed results.

Coverage data is collected per file using separate `.coverage` data files, which are later combined using `coverage combine` to produce an aggregate report. The framework generates both a textual coverage report (parsed to extract the overall coverage percentage) and an HTML coverage report for visual inspection. The coverage statistics include statements, missing lines, and the overall coverage percentage.

The iterative refinement loop is the defining innovation of the framework’s execution stage. When the initial raw tests fail pytest execution, the orchestrator captures the pytest error output, extracts the last 80 lines as a diagnostic excerpt, and passes this feedback to the refinement module. The `llm/test_refiner.py` module constructs a refinement prompt that includes the original source code, the failing test code, and the pytest error messages. The quantized model then generates refined tests that address the specific failures observed.

The orchestrator permits up to two refinement attempts per source file (configurable via `max_refine_attempts`). Each refinement iteration follows the same validation and execution cycle: the refined tests are post-processed, validated, executed with coverage measurement, and the results are evaluated. If the refined tests pass, the pipeline proceeds to the next file. If all refinement attempts are exhausted and the tests still fail, the framework invokes a decision handler that allows the user (or the automated decision logic in web mode) to choose between aborting the entire run, skipping the current file, or marking it as complete despite failures. This human-in-the-loop escalation mechanism ensures that the framework can continue processing remaining files without being blocked by pathological cases.

4.4 Workflow Architecture and Component Interaction

Figure 3 illustrates the complete workflow architecture, showing how the pipeline components interact with the web interface, the orchestrator backend, and the supporting infrastructure including the SQLite database, the execution environment, and the results dashboard.

The framework provides two entry points: a command-line interface (`run_pipeline.py`) for batch processing and a Flask-based web application (`app.py`) for interactive use. The web interface allows users to upload a Python repository as a ZIP archive, monitors progress in real time through a dashboard that polls the backend API, and provides control actions—abort, skip, and mark complete—that influence the orchestrator’s behaviour during execution. The CLI entry point provides the same pipeline functionality without the web overhead, accepting

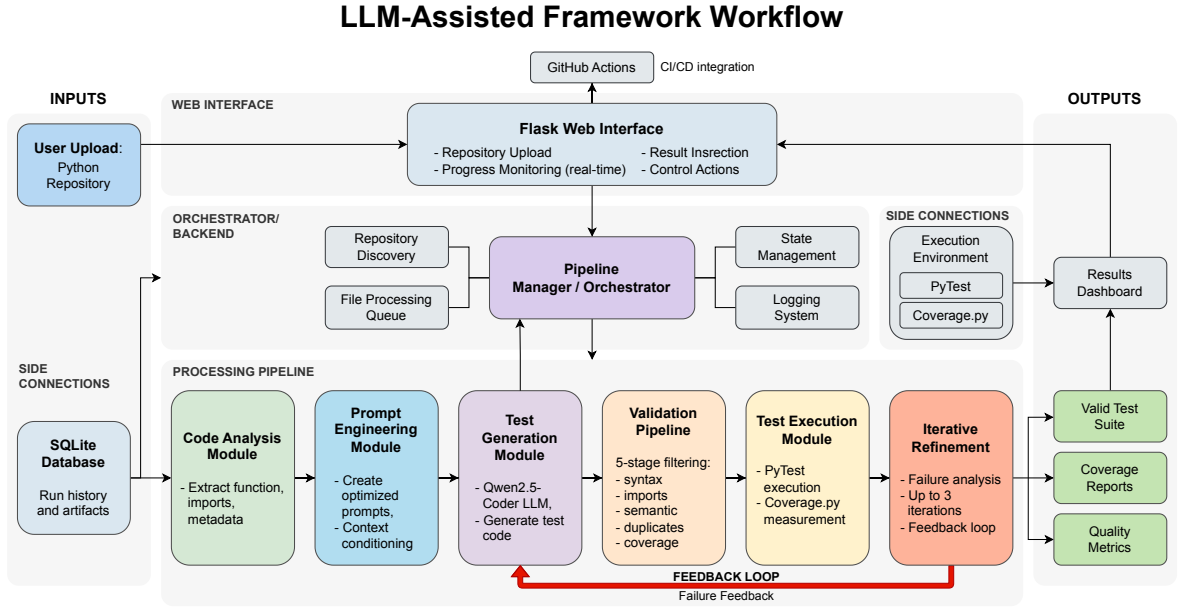


Figure 3: Workflow architecture and component interactions of the framework.

command-line arguments for the repository path, model path, output directory, and an optional file limit.

The `RepoTestOrchestrator` class in `pipeline/orchestrator.py` serves as the central coordinator. It loads the quantized model once at initialisation and reuses it across all file-processing iterations, avoiding the overhead of repeated model loading. The orchestrator maintains a structured output directory hierarchy for each run, comprising subdirectories for the evaluation sandbox, generated tests, coverage data, execution logs, HTML coverage reports, and per-file JSON result reports. This organisation ensures full traceability from input source files through intermediate artefacts to final results.

The event-driven architecture enables the web application to track progress without tight coupling to the pipeline internals. The orchestrator emits events at key lifecycle points: `run_started` when processing begins, `file_started` when a new source file enters the pipeline, `phase` events during each processing stage, `test_artifact` events when test files are produced, `file_completed` when processing finishes for a file, and `run_finished` when the entire run completes. The Flask application registers an event handler that persists these events to a SQLite database, enabling the dashboard to display real-time progress, logs, results, and generated test code.

4.5 Sequence of Operations

Figure 4 presents the sequence diagram detailing the temporal ordering of interactions between framework components during a complete test generation run.

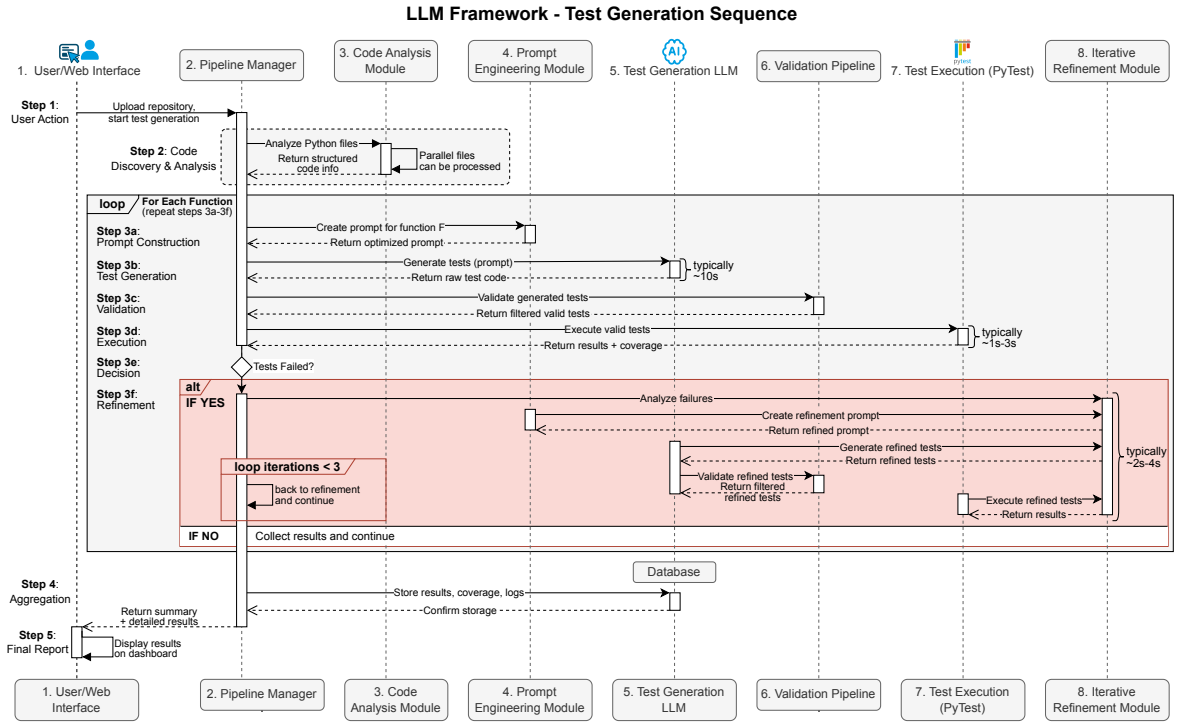


Figure 4: Sequence of interactions during test generation and refinement.

The sequence begins with the user uploading a repository through the web interface or invoking the CLI. The pipeline manager initiates code discovery and analysis, which returns structured code information for the discovered Python files. For each source file, the pipeline enters a loop that progresses through prompt construction, test generation via the LLM, validation of the generated tests, and execution with coverage measurement. If tests fail, the sequence branches into the refinement path, where failure analysis produces a new refinement prompt, the LLM generates refined tests, and the validation-execution cycle repeats. This loop continues for up to three iterations (one initial generation plus two refinements) before results are aggregated and stored to the database. The final step returns the summary and detailed results to the user interface for display on the dashboard.

4.6 Data Flow Through the System

The data flow through the framework follows a clearly defined transformation chain. Source code enters the system as raw Python files within a repository. The discovery module filters these files and produces a list of candidate paths. Each candidate is copied into the sandbox, producing a sandboxed module with a deterministic import path.

The sandboxed source code is preprocessed by stripping comments while retaining docstrings, yielding a cleaned code string. This string is interpolated into the prompt template along with the import path, producing a formatted prompt string. The prompt is tokenised by the model's

tokeniser and processed through the quantized Qwen2.5-Coder model, which generates a token sequence that is decoded back into a text string containing the raw test code.

The raw test code passes through the post-processing chain: code block extraction, header enforcement, truncation salvage, assertion splitting, and placeholder removal. The result is a syntactically valid Python file containing one or more test functions. This file is written to disk and passed to the pytest executor, which produces a pass/fail exit code, a textual output log, and a binary coverage data file. The coverage module parses the textual report to extract the coverage percentage and detailed statistics.

If the tests fail, the pytest output is truncated to the most relevant 80 lines and combined with the source code and existing test code to form a refinement prompt. This prompt re-enters the LLM, and the cycle repeats. The final outputs for each file include the raw test file, the refined test file (if refinement occurred), the per-file coverage data, and a JSON report containing all metrics. At the run level, the framework produces a combined coverage report, an HTML coverage visualisation, and a comprehensive JSON report summarising the entire run.

4.7 Technical Innovations and Design Rationale

Several technical design decisions distinguish this framework from existing approaches to LLM-based test generation.

The choice to employ a quantized 1.5-billion-parameter model rather than a larger commercial model represents a deliberate trade-off between capability and deployability. While larger models demonstrate superior performance on general code generation benchmarks, the domain-specific fine-tuning applied to Qwen2.5-Coder-1.5B-Instruct on the `kjain14/testgeneval` dataset—comprising 15,000 training examples of Python source code paired with corresponding PyTest test suites—narrows the capability gap for the specific task of test generation. The QLoRA training regime, which freezes the base model weights and trains only 4.36 million LoRA adapter parameters (0.28% of total parameters) targeting the attention projection matrices (`q_proj`, `k_proj`, `v_proj`, `o_proj`), enables fine-tuning on a single consumer GPU. The subsequent conversion to GGUF format with Q4_K_M quantization compresses the model from approximately 3.1 gigabytes (float16) to 941 megabytes, enabling CPU-only inference with acceptable latency.

The single-assertion-per-test enforcement is another distinctive design choice. Many LLM-based test generators permit the model to produce multi-assertion test functions, which conflates independent test objectives and obscures the diagnostic value of test failures. By applying AST-level splitting, the framework guarantees that each generated test function

verifies exactly one behavioural property, resulting in test suites that are more granular, more maintainable, and more informative when failures occur.

The feedback-driven refinement loop addresses a fundamental weakness of single-shot LLM generation: the model cannot observe the runtime behaviour of its own output. By capturing pytest execution results and feeding them back to the model as contextual evidence, the framework enables a form of self-correction. The refinement prompt explicitly instructs the model to diagnose faulty imports, fix them or remove them, and improve edge case coverage, creating a directed repair process rather than blind regeneration.

The sandboxing mechanism ensures isolation between the testing framework and the project under test. By copying source files into a controlled package namespace, the framework avoids import path collisions that commonly arise when test files attempt to import modules from arbitrary project structures. This approach also ensures that the original repository is never modified, preserving the developer’s working state.

Finally, the event-driven architecture and dual-interface design (CLI and web) reflect the framework’s intended role as both a research tool and a practical development utility. The web dashboard provides real-time visibility into the generation process, enabling researchers to observe model behaviour, inspect generated tests, and intervene when the model struggles with particular source files. The CLI interface supports batch processing for large-scale experiments, enabling the systematic evaluation of the framework across diverse repositories.

5 IMPLEMENTATION

This chapter details how the proposed framework is implemented in practice. The design objective is not merely to demonstrate that a language model can produce test code, but to build an engineering workflow that can be executed repeatedly on local hardware, inspected at file level, and controlled when automatic generation fails. The implementation therefore combines model inference, deterministic processing logic, persistent artefact storage, and an interactive control layer. All components are implemented in Python and integrated as a single repository-level system.

5.1 Implementation Scope and Module Structure

The codebase is divided into three implementation layers that correspond to the runtime responsibilities of the framework. The first layer is the model interaction layer in `llm/`, which loads the quantized model and applies generation and refinement prompts. The second layer is the orchestration layer in `pipeline/`, which handles file discovery, sandbox preparation, execution, coverage collection, refinement loops, and run-level aggregation. The third layer is the interface layer, implemented by a command-line entry point (`run_pipeline.py`) and a web application (`app.py`) used for interactive control and artefact browsing.

This separation is intentional. LLM output quality is uncertain by nature, but the surrounding mechanics of execution, reporting, and reproducibility should remain deterministic. By isolating LLM calls from pipeline control, the framework can retain predictable behaviour even when model outputs vary in quality.

5.2 Technology Choices and Runtime Dependencies

The runtime stack is intentionally compact: Flask for the web interface, pytest for test execution, Coverage.py for statement-level measurement, and `llama-cpp-python` for local inference against a GGUF model. A compact dependency set lowers the barrier to replication and simplifies environment management.

The deployed model is a fine-tuned variant of Qwen2.5-Coder-1.5B-Instruct, quantized to GGUF Q4_K_M. Inference is performed through `llama.cpp` bindings with configurable context length, thread count, and token limits. This choice directly supports the thesis constraint of local execution without reliance on cloud inference APIs.

The implementation runs in Python 3.10+ and is compatible with CPU-only execution. The model file is loaded once per run and reused across files, which reduces repeated initialisation overhead and stabilises per-file processing latency.

5.3 Model Adaptation and Packaging Workflow

Although this chapter focuses on system implementation, the deployed model is a critical component and its preparation affects runtime behaviour. The fine-tuning workflow is implemented in `llm_qwen_fine_tuning.ipynb` and follows a reproducible sequence: data loading, preprocessing, parameter efficient adaptation, merge, and quantization.

The model is trained on examples derived from TestGenEval, where each training item pairs source code with target test code. Preprocessing removes hash comments while preserving syntax-bearing text, then clips long examples to fit token limits. The resulting instruction format mirrors inference prompts so that the model is adapted to the same interaction pattern used at runtime.

QLoRA is used to reduce training cost while preserving adaptation quality. Attention projection modules are targeted with low-rank adapters, and only adapter parameters are updated. After training, adapters are merged into the base model and converted to GGUF. Q4_K_M quantization is then applied to produce a compact file suitable for CPU inference. The resulting model is stored under `models/` and referenced by pipeline configuration.

This packaging workflow provides two implementation advantages. First, the runtime deployment is simple because inference depends on a single GGUF file. Second, model replacement is straightforward: if a new fine-tuned variant is produced, the pipeline can reuse the same inference and orchestration code.

5.4 Inference Layer Implementation

The inference layer contains three operational responsibilities: model loading, prompt composition, and output normalization.

5.4.1 Model Loading and Invocation

The `LoadModel` abstraction in `llm/LLM_models.py` wraps model construction and chat completion calls. The constructor validates model availability and initialises a `Llama` instance with selected parameters such as context size, token batch size, and CPU thread count. The wrapper exposes a unified chat method so generation and refinement modules can invoke the model through a consistent interface.

A practical implementation detail is lifecycle scope: the model object is created once at orchestrator startup and reused for all file-level tasks. Without reuse, repeated model initialisation would dominate runtime for small target files and distort throughput measurements.

5.4.2 Prompt Templates

Prompt definitions are centralised in `llm/prompt_templates.py`. Two templates are maintained: one for initial test generation and one for refinement after pytest failure. The generation template enforces PyTest-style output constraints and inserts source code plus sandbox import path. The refinement template adds the failing test code and a bounded pytest excerpt.

Template centralisation has an engineering benefit beyond readability: experiments can change prompt wording while keeping orchestration logic fixed. This supports controlled prompt iteration and keeps prompt changes auditable in version control.

5.4.3 Raw Output Normalization

The model response is not assumed to be directly executable. The generation and refinement modules therefore apply a deterministic normalization chain before a candidate test file is accepted.

First, code extraction identifies a fenced Python block when present; otherwise the raw text is used. Second, import header enforcement guarantees `import pytest` and the expected sandbox import statement. Third, truncation salvage attempts to recover syntactically valid Python when model output is cut off. Fourth, assertion normalization transforms multi-assert test functions into one-assert-per-test form. Fifth, placeholder elimination removes vacuous tests such as `assert True`.

During refinement, an additional AST-level deduplication pass removes structurally duplicate tests. This reduces repeated failure cascades caused by the model emitting near-identical test functions with minor renaming.

5.5 Repository Orchestration

The class `RepoTestOrchestrator` in `pipeline/orchestrator.py` is the control center of each run. It creates a timestamped run directory, initialises metadata, coordinates file-level processing, and writes final reports.

5.5.1 Run Directory Layout

Each run directory contains a stable artefact schema: sandboxed source modules, generated tests, coverage files, logs, file-level JSON reports, and a combined HTML coverage report. This fixed layout is important for reproducibility and post-run analysis. Supplementary scripts and appendix chapters can consume artefacts without custom per-run parsing logic.

5.5.2 Discovery and File Selection

Python files are discovered recursively while excluding directories unrelated to target source logic, such as caches, virtual environments, build artefacts, and existing test directories. Lightweight file filters remove likely non-target files, including empty modules and common scaffolding files.

The resulting files are copied into an `eval_sandbox` package and renamed deterministically (`mod_0001`, `mod_0002`, etc.). This stabilises import paths across runs and prevents accidental dependency on the original repository import graph.

5.5.3 Per-file Processing Loop

For each sandbox module, the orchestrator executes a fixed sequence: generate raw tests, run pytest with coverage, refine on failure up to a configured limit, and persist the final file-level outcome. At each transition, the orchestrator emits structured events used by the CLI or web layer.

This loop is engineered to preserve traceability. The raw test file is always saved, refined files are versioned by attempt index, and each execution writes logs and measured coverage statistics. Even when a file fails, the run retains all intermediate artefacts for later diagnosis.

5.5.4 Decision Handling After Refinement Exhaustion

When maximum refinement attempts are exhausted and tests still fail, the orchestrator delegates to a decision handler. In web mode, the user can choose to abort the run, skip the file, or mark it complete. The selected action is stored in per-file and run-level reports. This mechanism prevents complete run failure due to a small number of difficult modules and aligns with repository scale operation.

5.6 Execution and Coverage Measurement

The module `pipeline/coverage.py` encapsulates subprocess execution of pytest and Coverage.py. For each candidate test file, the command path is deterministic: coverage is run

against a single sandbox module source target, pytest is executed in quiet mode, and failure short-circuiting is enabled via `-maxfail=1`.

Coverage output is parsed to extract statement counts and percentage values. These values are saved per file and later aggregated. Aggregation uses `coverage combine` over per-file data files, followed by textual and HTML report generation. The HTML report path is recorded in the run report and exposed through the dashboard.

The execution utility in `pipeline/utls.py` supports cooperative interruption. In polling mode, subprocesses are checked periodically against abort flags. If an abort is requested, the subprocess is terminated and the result is recorded with interruption markers. This behaviour is essential for responsive web control during long-running runs.

5.7 Web Application and Persistence Layer

The Flask application implements project upload, run orchestration, live monitoring, and artefact retrieval. Uploaded archives are extracted to a temporary workspace, normalised to a project root, and linked to a run record in SQLite.

The database schema stores run metadata, logs, per-file outcomes, and test artefact references. This schema enables dashboard reconstruction from persisted state instead of in-memory process state only. As a result, run history remains available even after process restarts.

A background thread executes orchestrator runs. Event callbacks translate orchestrator events into database updates. Status endpoints expose current progress, while control endpoints dispatch abort, skip, and mark-complete actions to a thread-safe control object. Generated tests and coverage reports are served through read-only endpoints for inspection.

This architecture intentionally separates long-running execution from HTTP request handling. The web server remains responsive while the pipeline performs model inference and subprocess execution.

5.8 Command-line Execution Path

The command-line interface in `run_pipeline.py` is kept minimal so the same orchestrator can be used in scripted workflows. Arguments support repository selection, model path, output directory, optional file limits, and verbosity controls. In CLI mode, run status and decisions are handled through console prompts and logs, preserving parity with web-mode functionality while remaining suitable for local experimentation.

5.9 Implementation Challenges and Engineering Responses

Several engineering challenges emerged during implementation.

The first challenge was incomplete model output at token boundaries. The response was a salvage strategy based on progressive truncation and AST parse checks. This does not recover omitted assertions, but it reliably converts many otherwise unusable outputs into executable candidates.

The second challenge was unstable imports in generated tests. The response was strict header enforcement with prolog-aware insertion, ensuring import statements are added without violating Python file preamble conventions.

The third challenge was test-quality noise: placeholders, duplicate tests, and multi-assert functions that reduced diagnostic clarity. AST-based filtering, deduplication, and one-assert normalization were introduced as deterministic correctives.

The fourth challenge was user control over long runs. The response was a cooperative interruption model used by both LLM calls and subprocess wrappers, combined with a condition-based decision channel for skip and mark-complete actions.

The fifth challenge was result transparency. The response was strict artefact persistence at run and file levels, including raw and refined tests, coverage data, logs, and structured reports. This supports both quantitative evaluation and manual audit.

5.10 Summary

The implementation realises the proposed framework as a complete local testing system rather than an isolated model demo. The final system integrates a fine-tuned quantized model, deterministic normalization logic, per-file execution and refinement, robust reporting, and dual interfaces for automated and interactive usage. The resulting artefact pipeline enables reproducible evaluation and supports the central thesis claim: LLM-assisted test generation is practically deployable when model output is embedded in a controlled execution, validation, and reporting architecture.

6 EXPERIMENTAL SETUP

This chapter specifies the concrete setup used to evaluate the implemented framework. The purpose is to make the experimental conditions transparent and reproducible, and to separate engineering facts from later interpretation. All settings reported here are grounded in artefacts contained in the repository: the fine-tuning notebook, pipeline source files, dependency manifests, and the stored run outputs under `experiments/`. No additional hidden configurations or undocumented datasets were used.

6.1 Dataset: TestGenEval

Model adaptation is based on TestGenEval, a public benchmark hosted on the Hugging Face Hub as `kjain14/testgeneval`. The dataset provides paired Python source code and test code examples and is designed for unit-test generation research. Its use in this work is motivated by direct task alignment with the deployed framework, which also targets Python unit test synthesis.

6.1.1 Dataset Origin and Composition

TestGenEval is derived from SWE-Bench-style repository data and includes multiple projects with realistic code and test conventions. The full dataset contains train, development, and test splits. In this thesis, only the training split is used for model adaptation, because the primary goal is to produce a deployable local model rather than to report benchmark competition results.

Each row includes source and test fields plus repository metadata. The implementation consumes `code_src` and `test_src` as core signals, while metadata fields are retained for traceability. Table 4 lists the principal columns relevant to this setup.

The dataset splits contain 15,439 training examples, 210 development examples, and 1,210 test examples. The fine-tuning procedure in this thesis uses 15,000 examples from the training split. This subset size is chosen to balance adaptation quality with practical GPU time and storage limits while still covering broad project diversity.

Table 5 summarises the split composition.

Table 4: Principal columns of the TestGenEval dataset and their descriptions.

Column	Description
repo	Name of the source repository (e.g., <code>numpy</code> , <code>scikit-learn</code>)
base_commit	Git commit hash identifying the repository snapshot
version	Software version tag
instance_id	Unique identifier for each code–test pair
code_src	Full source code of the Python module under test
test_src	Complete test suite corresponding to the source module
code_file	File path of the source module within the repository
test_file	File path of the test file within the repository
local_imports	Import statements local to the module
baseline_covs	Baseline code coverage statistics
patch	Diff patch associated with the SWE-Bench instance
test_patch	Diff patch for the corresponding test changes
preds_context	Prediction context metadata
id	Numeric row identifier

Table 5: Composition of the TestGenEval dataset splits.

Split	Total Examples	Used for Training
Train	15,439	15,000
Dev	210	—
Test	1,210	—
Total	16,859	15,000

6.1.2 Rationale for Dataset Selection

The dataset is selected for four practical reasons. First, examples come from real repositories and therefore encode realistic coding styles and edge cases. Second, data is Python-centric and compatible with pytest-oriented generation, which matches framework scope. Third, public availability improves reproducibility. Fourth, prior benchmark use provides context for interpreting model behaviour in a broader research landscape.

These factors make TestGenEval appropriate for adaptation and stress-testing, while keeping the workflow grounded in publicly inspectable data.

6.2 Data Preprocessing

Raw rows are transformed before fine-tuning to improve token efficiency and align examples with runtime prompts. The preprocessing logic is implemented in the notebook and mirrors constraints used during inference.

6.2.1 Comment Stripping

Both source and target code are processed with comment stripping that removes hash-style comments while preserving executable structure and docstrings. This reduces token usage without discarding core behavioural hints.

6.2.2 Length Clipping

After stripping, source code is clipped to 3,500 characters and target test code to 2,500 characters. For long examples, clipping preserves both beginning and ending segments around an ellipsis marker to retain imports, declarations, and late-file logic.

6.2.3 Training Text Formatting

Each example is formatted as instruction plus source plus expected test output, matching deployment-time prompt shape. This alignment is important because the model must learn not only code patterns but also response discipline under the same prompt framing used later in the pipeline.

6.2.4 Tokenization and Label Masking

Training texts are tokenized with the Qwen tokenizer under ChatML formatting. Examples are encoded as system, user, and assistant turns. Only assistant tokens contribute to loss: prompt tokens are masked with the standard ignore index -100 .

Maximum sequence length is set to 1,024 tokens and right-side truncation is applied when needed. Rows with missing source or target text are excluded.

6.3 Experiment Environment

Two execution environments are used in this thesis. Fine-tuning is performed in a GPU-enabled cloud environment. Inference and evaluation are executed locally on CPU, matching the practical deployment objective.

Table 6: Hardware and software specifications for the fine-tuning and inference environments.

Component	Fine-Tuning Environment	Inference Environment
Platform	Google Colab (cloud)	Local workstation
GPU	NVIDIA L4 (23.8 GB VRAM)	None (CPU only)
CUDA Version	CUDA 12.6	—
PyTorch Version	2.9.0+cu126	—
Python Version	3.10+	3.10+
OS	Ubuntu Linux (Colab)	Windows 10
Fine-Tuning Libraries		
Transformers	4.55.0	—
PEFT	0.17.0	—
Accelerate	1.12.0	—
BitsAndBytes	0.49.1	—
Datasets	4.5.0	—
Inference and Pipeline Libraries		
Flask	—	3.1.0
pytest	—	7.4.3
Coverage.py	—	7.3.2
llama-cpp-python	—	Latest

6.3.1 Fine-Tuning Environment

Fine-tuning is conducted on Google Colab with a single NVIDIA L4 GPU (23.8 GB VRAM), CUDA-enabled PyTorch, and Hugging Face training libraries. Adapter-based training allows the base model to remain frozen while LoRA weights are learned, reducing memory pressure and training time. After training, model merge and GGUF conversion are performed in the same environment.

6.3.2 Inference Environment

Inference and repository evaluation run on a local Windows 10 machine with Python 3.10+ and no dedicated GPU. The quantized model is loaded through llama-cpp-python and executed with CPU threading. This setup is intentionally conservative to test feasibility in resource-limited settings.

6.4 Experiment Design

The evaluation is run at file granularity inside each repository run. Each candidate source file is processed independently through generation, normalization, execution, and optional refinement. File-level outcomes are then aggregated into run-level reports.

6.4.1 Evaluation Procedure

For each run, the orchestrator discovers Python source files, applies filters, maps selected files to deterministic sandbox module names, generates raw tests, executes pytest with Coverage.py, refines failing outputs up to two rounds, and writes per-file plus run-level reports. The full artefact path includes `tests/`, `logs/`, `coverage/`, `htmlcov/`, and `file_reports/`.

6.4.2 Evaluation Metrics

The measured outputs include statement coverage percentages and counts, execution outcome flags, raw and final test counts, refinement attempts used, and run-level completion status. These values are captured directly from pipeline reports and not reconstructed manually.

6.4.3 Controls and Baselines

The setup preserves both raw and final test artefacts. This allows direct observation of what refinement changes, including cases where refinement repairs mechanical errors and cases where semantic assertion mismatches remain.

6.5 Model and Pipeline Configuration Parameters

Model and pipeline behaviour are controlled by explicit parameters documented in Tables 7 and 8. These values are important for reproducibility because minor changes in decoding or refinement limits can alter run outcomes.

6.5.1 Fine-Tuning Hyperparameters

Fine-tuning uses a single-epoch QLoRA configuration with constrained sequence length and gradient accumulation. Table 9 reports the key values used in training and packaging.

The selected hyperparameters prioritize practical convergence under memory constraints. A single epoch is used to avoid prolonged training while still adapting the model toward test-generation style and syntax discipline.

Table 7: Model inference configuration parameters.

Parameter	Value	Rationale
Model	Qwen2.5-Coder-1.5B-Instruct	Domain-specific code generation model
Quantization	GGUF Q4_K_M	4-bit mixed precision for CPU inference
Model Size	941 MB	Compressed from 2,944 MB float16
Context Window (n_ctx)	3,072 tokens	Accommodates source code + prompt + output
CPU Threads (n_threads)	6	Balanced parallelism for typical hardware
Batch Size (n_batch)	256	Prompt processing efficiency
Chat Format	ChatML	Compatible with Qwen2.5 instruction tuning
Temperature	0.0 (pipeline)	Greedy decoding for reproducibility
Max Output Tokens	1,200	Empirically tuned to avoid truncation and repetition
Repetition Penalty	1.05	Mild penalty to prevent degenerate loops

Table 8: Pipeline configuration parameters.

Parameter	Value	Rationale
Max Refinement Attempts	2	Two refinement cycles per source file
Generation LLM Attempts	1	Single LLM call per generation phase
Refinement LLM Attempts	1	Single LLM call per refinement phase
Pytest Max Failures	1 (-maxfail=1)	Fast failure for rapid feedback
Pytest Verbosity	Quiet (-q)	Minimal output for parsing
Coverage Mode	Statement only	Branch coverage disabled
Coverage Exclusions	<code>__main__ guard, pragma: no cover, TYPE_CHECKING, NotImplementedError</code>	Standard non-testable patterns
Coverage Omissions	<code>tests/*, test_*.py</code>	Exclude test files from coverage measurement
Feedback Excerpt Length	80 lines (tail)	Most relevant error information
Output Format	JSON per-file + JSON aggregate	Full traceability

Table 9: Fine-tuning hyperparameters for the QLoRA training procedure.

Hyperparameter	Value
Base Model	Qwen2.5-Coder-1.5B-Instruct
Total Parameters	1,548,072,448
Trainable Parameters (LoRA)	4,358,144 (0.28%)
LoRA Rank (r)	16
LoRA Alpha (α)	32
LoRA Dropout	0.05
Target Modules	q_proj, k_proj, v_proj, o_proj
Base Quantization	NF4 (4-bit NormalFloat)
Compute Dtype	float16
Sequence Length	1,024 tokens
Training Examples	15,000
Epochs	1
Per-Device Batch Size	1
Gradient Accumulation Steps	8
Effective Batch Size	8
Learning Rate	2×10^{-4}
Optimiser	Paged AdamW 8-bit
Precision	fp16 mixed precision
Gradient Checkpointing	Enabled
Total Training Steps	1,875
Training Duration	$\approx$ 1 hour 54 minutes
Final Training Loss	$\approx$ 0.62

6.6 Post-Quantization Model Verification

After quantization, the model is validated with smoke-test prompts to confirm that output remains coherent and compatible with pipeline post-processing. This verification step reduces risk of deploying a corrupted or misconfigured model artifact.

6.7 Reproducibility Considerations

Reproducibility is supported by deterministic decoding settings in the standard pipeline path, public dataset sourcing, pinned runtime dependencies, and rich run artefacts. Each run stores enough structured information to reconstruct which files were tested, how tests were generated, which refinements occurred, and what coverage and execution outcomes were observed.

Sandboxing further improves consistency by preventing direct dependence on a repository’s ambient import context. Deterministic file naming and stable processing order also reduce run-to-run variance.

One deliberate trade-off is `-maxfail=1`. It accelerates feedback loops for refinement but may stop execution before all failing assertions are visible. This is acceptable for iterative repair and was kept constant across runs.

6.8 Summary

In summary, the experimental setup combines public training data, explicit model adaptation settings, a CPU-focused deployment target, deterministic orchestration, and structured artefact capture. The setup is sufficiently constrained to enable repeatable comparisons, while still realistic enough to expose practical limitations of LLM-assisted testing in repository workflows.

7 EVALUATION METRICS

This chapter defines the metric system used to evaluate the implemented LLM-assisted testing framework. The framework generates tests per source file, executes them in an isolated sandbox, and records all outcomes in structured run artefacts. The purpose of the metric design is to evaluate practical performance under real pipeline constraints rather than idealized benchmark-only conditions.

The evaluation uses four complementary dimensions. The first is *executability*: generated tests should run under pytest and produce an interpretable result. The second is *structural reach*: generated tests should execute target program statements. The third is *refinement cost*: when initial tests fail, the framework should recover with limited repair cycles. The fourth is *suite compactness*: generated tests should remain concise enough for maintenance and diagnosis.

No single metric can capture all of these dimensions. Coverage alone does not prove semantic correctness. Pass rate alone does not indicate behavioural breadth. Refinement counts alone do not reveal whether final tests are reliable. For this reason, the chapter defines a combined metric set that can be computed from stored artefacts and interpreted jointly.

7.1 Measurement Context

All quantitative values are produced by the implemented pipeline and stored in JSON reports under the run directory. File-level records include execution exit code, coverage percentage and counts, refinement attempts used, raw and final test counts, and status flags for skip and mark-complete actions. Run-level reports aggregate these file-level outcomes and add combined coverage statistics.

Measurement is performed at file granularity. Each target Python file is copied into the `eval_sandbox` package and imported through a deterministic module alias (for example, `mod_0002`). Generated tests are then run only against that sandboxed module. This design reduces confounding effects from cross-module imports and allows direct attribution of outcomes to one source file at a time.

7.2 Coverage Metrics

7.2.1 Per-file Statement Coverage

The primary effectiveness metric is statement coverage from Coverage.py. Coverage is computed during pytest execution with the source scope restricted to the active sandbox module. Let S_{tot} be executable statements and S_{miss} be missed statements. Covered statements are

$$S_{\text{cov}} = S_{\text{tot}} - S_{\text{miss}}. \quad (7.1)$$

Per-file statement coverage is

$$C_{\text{stmt}} = \frac{S_{\text{cov}}}{S_{\text{tot}}} \times 100\%. \quad (7.2)$$

The framework stores both percentage and denominator statistics. This is important for interpretation: in small modules, a single missed statement can change the percentage substantially.

Coverage is configured in a generated `.coveragerc` file. Branch coverage is disabled in this thesis to keep the main indicator stable and comparable across files. Standard exclusions are applied (for example, `pragma: no cover`, main guards, and non-target patterns), and test files are omitted from measured sources.

7.2.2 Run-level Overall Coverage

After file-level execution, coverage data files are combined and a run-level TOTAL is produced. This aggregate is stored as `overall_coverage_total_percent`, with supporting counts in `overall_coverage_stats`. Run-level coverage summarizes structural reach across all processed files in one experiment run.

Coverage is interpreted as a structural metric. It shows how much code was executed, not whether all expected outputs are semantically correct. Therefore, coverage is always analysed alongside execution outcomes.

7.3 Execution Outcome Metrics

7.3.1 Strict Pass and Operational Completion

Each evaluated file has a pytest `exit_code`. A zero exit code indicates strict pass under the generated test file. Non-zero indicates failure or error. In addition, the framework records operational actions such as `skipped_by_user`, `skipped_due_to_failures`, and `marked_complete`.

Because interactive operation is part of the implemented system, two success rates are reported.

Strict pass rate:

$$R_{\text{pass}} = \frac{N_{\text{pass}}}{N_{\text{tested}}}. \quad (7.3)$$

Operational completion rate:

$$R_{\text{complete}} = \frac{N_{\text{pass}} + N_{\text{marked}}}{N_{\text{tested}}}. \quad (7.4)$$

Here, N_{tested} is the number of processed non-skipped files, N_{pass} is strict passes, and N_{marked} is files accepted as complete after operator decision. Reporting both avoids conflating autonomous success with managed continuation.

7.3.2 Skip and Failure Rates

Two additional rates describe non-success outcomes:

$$R_{\text{skip}} = \frac{N_{\text{skip}}}{N_{\text{tested}}}, \quad R_{\text{skipfail}} = \frac{N_{\text{skipfail}}}{N_{\text{tested}}}. \quad (7.5)$$

N_{skip} counts user-initiated skips and N_{skipfail} counts files skipped after repeated failed refinement attempts. These measures help distinguish operational workflow choices from intrinsic generation limitations.

The framework also stores a tail excerpt of pytest output per file. Although not a scalar metric, these excerpts support qualitative failure analysis (for example, assertion mismatch versus import issues).

7.4 Refinement Metrics

Iterative repair is quantified by `refine_attempts_used`. In the current configuration, at most two refinement rounds are applied per file. A value of zero means the raw generated tests were accepted without repair.

Mean refinement effort is

$$\bar{A} = \frac{1}{N_{\text{tested}}} \sum_{i=1}^{N_{\text{tested}}} A_i, \quad (7.6)$$

where A_i is refinement attempts used for file i .

A complementary indicator is the share of files that required any refinement, $P(A_i > 0)$. Together, these values describe how often feedback is necessary and how expensive the repair process is within the configured budget.

The refinement prompt uses a bounded pytest excerpt instead of full logs, trading completeness for stable token usage and focused diagnostics.

7.5 Test Suite Size Metrics

The framework records raw and final test-function counts as `raw_tests` and `final_tests`. Counts are computed with AST parsing of top-level functions named with the `test_` prefix, avoiding string-based counting errors.

Mean final suite size is

$$\bar{T}_{\text{final}} = \frac{1}{N_{\text{tested}}} \sum_{i=1}^{N_{\text{tested}}} T_{\text{final}}^{(i)}. \quad (7.7)$$

Refinement-induced size change for file i is

$$\Delta T^{(i)} = T_{\text{final}}^{(i)} - T_{\text{raw}}^{(i)}. \quad (7.8)$$

Suite-size metrics are interpreted as context signals. Very small suites may under-sample behaviours, while large suites can indicate redundancy. The values are most useful when analysed together with coverage and pass outcomes.

7.6 Structural Quality Signals

Beyond numeric aggregates, the implementation includes deterministic quality controls that directly affect measurable outcomes:

- syntax validation and truncation salvage using `ast.parse`;
- enforced import headers for pytest and sandbox module access;
- removal of placeholder tests (for example, `assert True`);
- AST-based deduplication in refinement;
- one-assert-per-test normalization for improved failure localization.

These controls are not reported as standalone scalar scores, but they improve the trustworthiness of measured outcomes and reduce noise in downstream analysis.

7.7 Metric Summary

Table 10 summarizes the metrics used in this study.

Table 10: Summary of evaluation metrics and their intended interpretation.

Metric	Definition (implementation)	Interpretation
Statement coverage C_{stmt}	Coverage.py statement percentage with stored statement/missing counts per file.	Structural reach of generated tests in one module.
Overall coverage	Combined run-level coverage after <code>coverage combine</code> .	Aggregate structural reach over a run.
Strict pass rate R_{pass}	Fraction of tested files with pytest exit code 0.	Autonomous executability success.
Completion rate R_{complete}	Fraction of tested files that pass or are marked complete.	Operational throughput in interactive runs.
Skip-related rates	Fractions of user skips and failure-driven skips.	Workflow interruption and unresolved difficulty.
Refinement effort $\bar{A}$	Mean <code>refine_attempts_used</code> per tested file.	Repair cost after initial generation.
Test counts $\bar{T}_{\text{final}}, \Delta T$	AST-counted raw/final test functions and their delta.	Suite compactness and refinement side effects.
Pytest excerpt (qualitative)	Stored tail of pytest output per file.	Failure-type diagnosis and interpretation support.

In summary, the evaluation design combines structural, operational, and process metrics derived directly from stored artefacts. This combined view is necessary for LLM-assisted testing because high coverage, strict pass success, and low refinement effort are related but not equivalent properties.

8 RESULTS AND ANALYSIS

This chapter analyses the experimental outcomes produced by the LLM-assisted software testing framework. The analysis is grounded strictly in the artefacts generated by the implemented pipeline and stored under the project’s `experiments/` directory. Each recorded run includes a structured report (`repo_report.json`), file-level JSON summaries, generated test files (raw and refined), subprocess logs capturing `pytest` and `Coverage.py` output, and a combined HTML coverage report. These artefacts enable transparent and reproducible analysis of both quantitative performance (coverage, executability, refinement effort, and test suite size) and qualitative behaviour (failure modes and limitations).

The evaluation focuses on the framework’s ability to (i) generate syntactically valid and executable `pytest` test suites for sandboxed Python modules, (ii) exercise the code under test as measured by statement coverage, and (iii) recover from failures via iterative refinement using `pytest` feedback. Since the framework operates in a setting where ground-truth specifications are typically unavailable, the results must be interpreted carefully: high coverage indicates broad execution of statements, but it does not guarantee that the expected assertions are semantically correct. The following sections therefore analyse coverage in conjunction with execution outcomes and observed failure patterns.

8.1 Overview of Recorded Experimental Runs

The recorded experimental evidence for this thesis consists of eight completed runs for which a `repo_report.json` is present. Across these runs, the framework processed 22 file-level evaluations (excluding user-skipped files). The runs represent repeated executions on small repositories or subsets thereof, including two-file runs (commonly `leapyear.py` and `simple.py`) and four-file runs that additionally include modules such as `moves2_3.py` and `recursionnnn.py`. The runs were executed using the same core pipeline stages described in Chapters 7–8 (generation, validation, execution with `Coverage.py`, and refinement), but some report fields vary slightly across runs due to incremental development and logging changes. This is visible, for example, in the presence or absence of certain diagnostic fields (e.g., `pytest_excerpt`) in the stored JSON.

Table 11 provides a run-level summary extracted directly from the run reports and aggregated into `experiments/summary_runs.json`. In addition to coverage, the table shows the strict `pytest` pass count (files that pass `pytest` without being marked complete), the number of files

marked complete, the number of files skipped after repeated failures, and the average number of refinement attempts used per file.

Table 11: Run-level summary of recorded experiment outputs.

Run	Files	Overall cov. (%)	Mean file cov. (%)	Pytest pass	Marked	Skipped (fail)	Avg. refine
Run 1	2	89.0	91.5	2	0	0	0.00
Run 2	4	94.0	95.75	3	0	1	0.25
Run 3	2	89.0	91.5	2	0	0	0.00
Run 4	4	94.0	95.75	3	1	0	0.50
Run 5	2	89.0	91.5	2	0	0	0.00
Run 6	4	94.0	95.75	3	1	0	0.50
Run 7	2	89.0	91.5	1	0	0	0.50
Run 8	2	89.0	91.5	2	0	0	0.00

Two immediate observations can be made. First, the overall statement coverage across runs is consistently high (89% or 94%), with the difference attributable to the number of files and the corresponding number of executable statements included in the combined coverage report. Second, the framework achieves a high strict pytest pass count for most runs; however, certain modules repeatedly trigger assertion-level mismatches that persist even after refinement, leading either to skipping after repeated failures or to explicit “marked complete” outcomes to allow the run to proceed.

8.2 Aggregate Performance Across All Files

To characterise overall behaviour rather than individual run idiosyncrasies, aggregate statistics were computed across all non-skipped file evaluations (22 files) using the stored run reports. The aggregate summary is stored as `experiments/summary_aggregate.json`. The analysis below reports only values derived from these artefacts.

Across the 22 evaluated files, the mean per-file statement coverage is 93.82%, with a median of 100%. The minimum observed per-file coverage is 83% and the maximum is 100%. The distribution is therefore heavily skewed toward full coverage for the small modules evaluated, with a minority of cases consistently leaving one executable statement uncovered.

In terms of executability, 18 out of 22 files produce a strict pytest pass (81.8%). In addition, two files are recorded as `marked_complete`, which the framework treats as an operator-accepted completion state (typically used when refinement attempts are exhausted but the run should continue). If marked-complete files are included as completed outcomes, 20 out of 22 files reach a completed state (90.9%). The remaining cases correspond to files that fail pytest without being marked complete.

Refinement is used sparingly: 4 out of 22 files (18.2%) require at least one refinement attempt. The mean number of refinement attempts used per file is 0.27, and the observed refinement counts are concentrated at zero (18 files), with only a small number of files requiring one (2 files) or two refinement iterations (2 files). This indicates that, for the evaluated modules, the framework frequently generates runnable tests in the first attempt, but a small subset of modules triggers recurring semantic or oracle-related failures.

Finally, test suite size remains modest. Summed across all evaluated files, the raw generation stage produces 35 test functions, and the final stage produces 36 test functions. The mean number of tests per file is therefore 1.59 before refinement and 1.64 after refinement. These values are consistent with the pipeline’s emphasis on generating compact, per-file test suites rather than exhaustive combinatorial input exploration.

8.3 Coverage Outcomes and Interpretation

8.3.1 Why overall coverage appears as 89% vs. 94%

The combined statement coverage reported at the run level depends on the number of executable statements included in the run and how many remain uncovered. In the recorded runs, two common totals appear in the Coverage.py summary:

- Runs with two evaluated modules report 9 executable statements in total and 1 missing statement, yielding 89% overall coverage.
- Runs with four evaluated modules report 18 executable statements in total and 1 missing statement, yielding 94% overall coverage.

This pattern reveals an important property of statement-level aggregation: when the total number of executable statements is small, a single missed statement produces a visibly large decrease in the overall percentage. Consequently, the difference between 89% and 94% in these experiments does not necessarily imply a qualitative change in the generator’s ability, but rather reflects the constant “one missed statement” pattern distributed across different totals.

8.3.2 Per-file coverage patterns

The coverage distribution suggests that most evaluated modules are fully exercised by the generated tests, while one recurrent module consistently achieves 83% coverage due to one uncovered statement. Inspection of the stored artefacts shows that this gap typically corresponds to a missing test case for the “normal” execution path of a function when the model focuses on the exceptional path (e.g., testing division-by-zero behaviour but not testing a successful division). In practical terms, this indicates that the LLM can often identify and

test edge cases, but it may under-sample ordinary return paths unless prompted or refined to do so.

Table 12 illustrates this pattern using a representative four-file run. The table is extracted from one recorded `repo_report.json` and shows that three modules reach 100% statement coverage, while one module remains at 83% coverage. The same run also demonstrates a key caveat: a module may achieve full coverage while still containing failing assertions, underscoring why coverage must be interpreted alongside pass/fail outcomes.

Table 12: Representative file-level outcomes from a recorded four-file run.

Module	Final tests	Refine	Coverage (%)	Outcome
leapyear.py	1	0	100	PASS
moves2_3.py	3	2	100	MARKED_COMPLETE
recursionnnn.py	1	0	100	PASS
simple.py	2	0	83	PASS

The results in Table 12 highlight two central findings. First, statement coverage is frequently maximised for small modules, indicating that the generated tests can reach most executable statements. Second, full coverage does not necessarily coincide with a strictly passing test suite, particularly when the LLM generates incorrect expected values (oracle errors). The next section analyses this executability dimension in more detail.

8.4 Executability and Correctness Outcomes

8.4.1 Strict pass rate vs. operational completion

The framework reports the pytest exit code for each file and records additional status flags to support interactive run management (skip and mark-complete actions). This enables two complementary interpretations of “success.” The strict interpretation considers only true pytest passes, whereas the operational interpretation considers a file “complete” if it either passes pytest or is marked complete by the operator after refinement is exhausted.

In the recorded experiments, the strict pass rate is 81.8% (18/22). When marked-complete outcomes are included as completed states, the completion rate rises to 90.9% (20/22). This gap is informative: it indicates that a small subset of files repeatedly exhibits semantic mismatches that are difficult to resolve automatically within the configured refinement budget, but the framework can still complete repository-scale processing without stalling through a human-in-the-loop decision mechanism.

8.4.2 Nature of failures: oracle errors dominate

The dominant observed failure mode in the stored logs is assertion failure due to incorrect expected values. Two concrete examples demonstrate this pattern.

Leap year edge case (century rule). In one recorded run on a small project, raw generation produced a test file where zero tests were collected (`raw_tests=0`), i.e., only the import header was present and no `test_` functions were defined. After one refinement step, the model produced a single test function containing many assertions over a range of years. The test suite failed on the case year 2100, where the model asserted that 2100 is a leap year. The implementation under test follows the correct leap year rule (divisible by 100 but not by 400 implies not a leap year), and therefore returns `False`. This mismatch indicates that the test oracle—the expected value encoded by the generated assertion—was incorrect, even though the function under test behaved correctly and the test invoked the correct API. Coverage remained 100% for the file, illustrating that execution reach can be high even when assertions are semantically wrong.

Ambiguous specification in a numerical helper. Another recurring failure is observed in a module implementing `calculate_min_moves(n)`. The function’s docstring states that it computes minimum moves to reach a point using steps of 2 or 3, but the implementation returns `ceil(n/3)` with a special case for $n = 1$. The LLM-generated tests assumed an alternative interpretation and asserted different expected outputs (e.g., asserting that the result for $n = 3$ should be 2). Pytest therefore failed with an assertion error, while coverage reported 100%. This case illustrates a fundamental limitation of LLM-based test generation: when the source code’s semantics are non-obvious or the documentation is ambiguous or inconsistent with the implementation, the model may generate plausible but incorrect oracles.

Importantly, the stored experimental evidence does not show a large number of failures due to syntax errors or missing imports. This aligns with the framework’s implementation, which enforces a valid module import header and uses AST-based parsing and truncation salvage. Consistent with this, the aggregate failure categorisation computed from available excerpts records zero `SyntaxError` and zero `ImportError/ModuleNotFound` cases; some failures remain labelled as “Other/Unknown” because not every run report contains a structured pytest excerpt for automated classification. While the absence of syntax/import failures in these small runs cannot be generalised to arbitrary repositories, it indicates that the pipeline-level safeguards are effective in reducing common mechanical generation errors.

8.5 Effect of Iterative Refinement

8.5.1 When refinement helps

The refinement loop is designed to improve executability by conditioning regeneration on the observed pytest output. In the recorded runs, refinement is invoked for 4 files, indicating that most files either pass on the first attempt or fail in a way that is not resolved within the available refinement budget.

In the best case, refinement converts an unusable raw output into an executable test suite. The leap year example demonstrates that raw generation can sometimes produce a test file with zero collected tests (`raw_tests=0`), which is not useful for evaluation. Refinement then generates a substantive test suite, demonstrating that the error-conditioned prompt can stimulate the model to produce test functions even when the initial generation did not. This reflects a practical advantage of the framework: the pipeline can detect that zero tests were collected and attempt recovery instead of silently accepting an empty suite.

8.5.2 When refinement does not help

The same leap year run also demonstrates a limitation: even after refinement produces executable tests, it may still encode incorrect assertions for domain-specific corner cases. In such cases, the refinement loop is constrained by the information present in the failure feedback. A pytest failure reveals that an assertion is incorrect, but it does not reveal which side—implementation or test—is wrong. Unless the model can infer the correct expected values from the code alone, it may preserve an incorrect belief and continue generating failing oracles.

Similarly, for the ambiguous `calculate_min_moves` module, refinement produces tests that are syntactically correct and execute successfully up to the failing assertion, but it does not resolve the semantic mismatch. This suggests that the refinement mechanism is effective at repairing mechanical issues (imports, syntax, missing tests) but less effective at correcting deep semantic misunderstandings when there is no explicit specification oracle.

8.5.3 Refinement budget and development variability

The recorded runs show refinement attempt counts of 0, 1, and 2. The current orchestrator implementation allows up to two refinement iterations, but the presence of runs where a file is marked as skipped after one refinement attempt indicates that the effective refinement budget or user decision path can vary across recorded experiments. This is not a methodological flaw; rather, it reflects the framework’s interactive design and iterative development during experimentation. For analysis, it implies that refinement performance should be interpreted

as conditional on the configured attempt budget and the decision policy (skip versus mark complete).

8.6 Test Suite Size and Structure

Generated test suite size in these experiments is small (typically 1–3 test functions per file). This is consistent with the framework’s emphasis on producing compact, focused unit tests suitable for rapid execution in a repository-scale pipeline. Small test suites reduce runtime, which is especially relevant because the pipeline runs `pytest` separately for each file under coverage measurement.

However, the stored artefacts also reveal that test structure can vary depending on the generation pathway. In the leap year refinement example, the final output includes many assertions within a single test function. This structure is less desirable for diagnosis, because one failing assertion causes the entire test function to fail and stops execution due to `-maxfail=1`. In contrast, the standard pipeline includes AST-based transformations that split multiple assertions into separate tests, improving failure localisation and aligning with the thesis’ design goal of one-assert-per-test. The presence of both structures in the recorded artefacts suggests that some runs were executed using a “fast” or alternative configuration that did not apply all normalisation steps uniformly. This variation should be considered when comparing test count and maintainability across runs.

Despite these structural differences, the overall test counts remain stable from raw to final output (35 to 36 total), implying that refinement is not primarily used to inflate test suite size. Instead, when refinement changes test count, it tends to do so by turning an empty or non-collected test file into a minimal, executable suite.

8.7 Observed Trends, Improvements, and Unexpected Behaviours

Several patterns emerge from the experimental artefacts.

High coverage on small modules is achievable and repeatable. The repeated runs on the same small module sets produce stable coverage totals and per-file coverage values. In particular, three-module subsets frequently reach 100% per-file coverage across all files, and even when overall coverage is below 100%, it is typically due to a single uncovered statement rather than widespread unexercised code.

Edge-case focus can leave normal paths uncovered. The recurrent 83% coverage result indicates a specific behavioural bias: the model often generates an exception-path test (e.g., division by zero) but omits a corresponding normal-path test (e.g., successful division). This is an important insight for future prompt engineering and refinement strategies. A simple

prompt-level addition (e.g., “include both normal and error cases for each function”) could potentially reduce this gap, but such modifications must be evaluated empirically.

Coverage and correctness diverge in oracle-driven failures. Modules such as `calculate_min_moves` demonstrate that coverage can reach 100% while `pytest` fails due to incorrect expected values. This divergence is expected in automated test generation, and it motivates the use of combined metrics: coverage measures reach, while pass rate measures executability and oracle consistency.

Interactive controls prevent run stalling. The presence of `marked_complete` and `skipped_due_to_failures` outcomes demonstrates that the framework can continue processing even when a file repeatedly fails. This is a practical benefit of the web-driven orchestration model, although it introduces a distinction between strictly passing test suites and operationally accepted completion.

An unexpected behaviour observed in one run is the generation of a raw test file from which zero tests were collected (`raw_tests=0`), which requires refinement to produce any substantive tests. This suggests that, in some cases, the model may fail to produce test functions at all under the raw prompt, potentially due to prompt length, token budget constraints, or model degeneracy. The pipeline’s ability to detect and recover from this condition is therefore an important robustness feature.

8.8 Errors, Failures, and Limitations

The experimental artefacts reveal both methodological limitations of the current evaluation and technical limitations of the framework.

8.8.1 Limitations of the recorded experiment set

The recorded runs evaluate a small number of files and a small number of distinct repositories. Consequently, the observed high coverage and pass rates should be interpreted as results for the specific evaluated modules rather than as general performance claims. Small modules can be fully covered with only a few tests, and a single missed statement can produce a large decrease in percentage terms. For stronger external validity, future evaluation should include larger repositories, more diverse code patterns (classes, I/O, third-party dependencies), and more complex test scaffolding requirements.

In addition, some report fields are absent in certain runs, indicating that the logging schema evolved during development. For example, not all file-level entries include `pytest_excerpt`. In such cases, failure classification must rely on the log files stored under `logs/`, which remain available for manual analysis but complicate fully automated categorisation.

8.8.2 Technical limitations of LLM-generated tests

The primary technical limitation observed is the prevalence of oracle errors for domain-specific edge cases and ambiguous specifications. The leap year failure at year 2100 demonstrates that even well-known rules can be misapplied by the model, resulting in failing assertions despite correct implementation. This is a significant limitation for autonomous test generation: without an external specification oracle or metamorphic relations, the system cannot reliably determine whether the test or the code is at fault.

Another limitation is that coverage-driven success can be overestimated when `pytest` is configured to stop after the first failure (`-maxfail=1`). When a test suite fails early, coverage still reflects statements executed up to the failure point, which can be high in small modules. Therefore, high coverage in a failing run does not imply a reliable or complete test suite, and the combination of coverage with pass/fail status is essential for truthful interpretation.

Finally, while the framework includes several safeguards against mechanical generation issues (syntax validation, header enforcement, truncation salvage), these mechanisms do not guarantee that the tests meaningfully capture intended program behaviour. They ensure executability and structural validity, but semantic quality remains dependent on the model and the available context.

8.9 Summary

The experimental results demonstrate that the proposed framework can reliably generate executable `pytest` tests for small Python modules and achieve high statement coverage in the recorded runs. Across eight recorded runs and 22 evaluated files, per-file coverage is high (mean 93.82%, median 100%), and the framework achieves an 81.8% strict `pytest` pass rate. Iterative refinement is required for a minority of files (18.2%), and interactive controls (skip and mark complete) enable repository-scale runs to complete without stalling on difficult modules.

At the same time, the results expose a critical limitation: coverage can be maximised while semantic correctness fails due to oracle errors. The observed failures are dominated by incorrect expected values in assertions for edge cases or ambiguous logic. This finding supports the broader thesis argument that LLM-assisted test generation is most effective when combined with execution feedback, robust validation, and transparent reporting, and that further research is needed to mitigate oracle errors through improved prompting, specification inference, or complementary analysis techniques.

9 DISCUSSION

This chapter interprets the experimental findings of the proposed LLM-assisted software testing framework and relates them to established work in automated test generation. The discussion is grounded in the recorded artefacts analysed in Chapter 9 (e.g., `repo_report.json`, logs, generated tests, and `Coverage.py` outputs) and focuses on what the outcomes imply for effectiveness, reliability, and practical deployment.

9.1 Interpretation of Effectiveness and Reliability

The results indicate that the framework can generate executable unit tests for small Python modules with minimal manual scaffolding. Across eight recorded runs and 22 file-level evaluations, the framework achieved a strict pytest pass rate of 81.8% (18/22) and an operational completion rate of 90.9% (20/22) when “marked complete” outcomes are included. The operational metric is practically important because the framework supports interactive orchestration: if refinement does not resolve a failure within the configured budget, an operator can accept the best available artefact and continue repository-scale processing.

Coverage outcomes further support the effectiveness of the generation stage in reaching the code under test. The observed per-file statement coverage has a mean of 93.82% and a median of 100%, with most modules reaching full coverage using only one to three generated test functions per file. At the run level (Table 11), overall coverage is consistently high (89% or 94%), which Chapter 9 attributes primarily to small statement totals where a single uncovered statement materially shifts the percentage.

The pipeline also shows strong mechanical reliability in the recorded runs. The aggregate failure categorisation derived from available excerpts contains zero `SyntaxError` and zero `ImportError/ModuleNotFoundError` cases, consistent with safeguards such as forced import headers, AST-based validation/normalisation (including truncated-output salvage), and deterministic sandbox imports under `eval_sandbox`.

Nevertheless, the reliability of generated tests as behavioural specifications is bounded by the “oracle problem” in software testing [4]. The observed failures in Chapter 9 are dominated by assertion mismatches (oracle errors), including incorrect expectations for a leap-year century edge case and disagreements with behaviour implied by ambiguous documentation. Consequently, the framework is strong at producing executable tests and exercising code paths, but weaker at guaranteeing semantic correctness of the assertions.

9.2 Comparison with Expectations and Prior Approaches

The results largely match expectations for an LLM-driven pipeline evaluated via statement coverage and executability. High coverage is easier to achieve on small, function-centric modules, making the median of 100% per-file coverage unsurprising. The more informative outcomes are the failure cases, which show that plausible tests can still fail on subtle domain rules or underspecified behaviour—consistent with broader findings that post-processing improves syntactic validity but does not eliminate semantic errors [7].

Compared with classical automated unit test generation tools, the proposed framework occupies a different point in the design space. Search-based approaches such as EvoSuite explore program behaviour to maximise coverage and synthesise assertions from observed executions [3], while feedback-directed random testing tools such as Randoop generate regression-style assertions based on observed outcomes [1]. In contrast, the present framework uses an LLM to propose tests directly from code context, then validates them through pytest execution and Coverage.py. This reduces the need for sophisticated analysis machinery, but it increases sensitivity to oracle errors when semantics are ambiguous.

Overall, the experiments support a hybrid interpretation: LLMs are useful for proposing scaffolding and candidate inputs, while execution feedback is essential for filtering and repair. The low refinement usage (4/22 files needing refinement; mean 0.27 attempts per file) suggests the pipeline is a strong first-pass test bootstrapper for small modules, but persistent oracle errors show that execution feedback alone cannot guarantee correct expected outcomes.

9.3 Strengths Demonstrated by the Framework

One demonstrated advantage is practicality: the framework produces small test suites (35 raw tests and 36 final tests across 22 files) that run quickly and integrate with standard pytest workflows. Per-file isolation (copying each module into `eval_sandbox` as `mod_XXXX.py`) also improves reproducibility by standardising imports and reducing dependence on repository-specific context.

Another strength is robustness through normalisation. The generation and refinement modules include guard rails tailored to LLM outputs (required imports, truncated-output salvage, placeholder removal, deduplication, and optional assertion-splitting), producing structured test artefacts that can be executed and evaluated consistently.

From a deployment perspective, the implementation also demonstrates a pragmatic pathway for applying LLM assistance without relying on external services: the pipeline loads a local GGUF model through `llama-cpp-python` and runs generation/refinement on the same host as the

testing workflow. This design supports offline experimentation and reduces concerns about transmitting proprietary code to third-party APIs. At the same time, local inference introduces resource constraints (e.g., bounded context and token budgets), which may contribute to behaviours such as truncated outputs or incomplete test suites; the framework’s salvage and validation steps are therefore not merely conveniences but necessary reliability mechanisms under realistic compute limits.

Finally, the interactive orchestration model (skip and mark-complete) is a workflow-level reliability mechanism: it preserves pipeline throughput when a small number of files are difficult to resolve automatically.

9.4 Limitations and Likely Causes of Performance Gaps

The most significant limitation evidenced by the experiments is semantic reliability of assertions. The leap-year example shows that a single incorrect expectation can invalidate an otherwise comprehensive test function, and the numerical helper example shows that tests can fail even with 100% coverage when the model’s interpretation diverges from the implementation. These outcomes are consistent with the oracle problem [4]: without an independent source of truth, an automated system cannot reliably determine whether the code or the generated test oracle is correct. Refinement helps when the failure is structural (e.g., zero collected tests), but it is less effective when the failure is conceptual.

Several additional constraints should be treated as threats to validity. The evaluation set is small and dominated by short modules, and coverage was measured as statement coverage (branch coverage disabled). In addition, pytest is invoked with `-maxfail=1`; when a suite fails early, coverage reflects only execution up to the failure point, which can appear deceptively high for small code. Finally, the artefacts show some configuration variability (e.g., `fast_mode`) and occasional missing structured excerpts, which complicates fully automated failure categorisation.

The current codebase also contains an implementation of mutation testing (via Cosmic Ray) as an additional adequacy signal, but mutation scores are not present in the analysed experimental artefacts. As a result, the discussion in this thesis is limited to executability, statement coverage, refinement effort, and the observed failure modes; future evaluations would benefit from exercising mutation testing systematically to better assess fault-detection capability beyond coverage.

For clarity, the limitations most directly supported by the current evidence are:

- **Oracle errors dominate failures:** incorrect expected values can persist even after refinement.

- **Normal-path gaps:** a recurrent 83% case indicates under-testing of ordinary return paths when edge cases are emphasised.
- **Limited external validity:** only 22 file evaluations across small modules; results should not be generalised to large repositories.
- **Metric scope:** statement coverage only; no mutation score is reported in the analysed runs despite a mutation-testing module existing in the codebase.

9.5 Practical Implications for Real-World Testing

Despite the limitations, the findings suggest realistic ways the framework could be used in practice. It is well-suited for test bootstrapping in early development or legacy modernisation: developers can generate a small initial suite, review assertions, and iteratively correct or extend tests where oracle errors occur. It can also act as a coverage-oriented exploration assistant: even when assertions are uncertain, generated tests expose executed paths, exception behaviour, and concrete call examples, and the reports highlight systematic gaps such as missing normal-path tests.

Overall, the experiments support a cautious but positive conclusion: LLM-assisted test generation can be practical for producing executable, coverage-improving tests on small modules when combined with execution feedback and robust normalisation. At the same time, semantic correctness remains the principal barrier to full automation, and human review (or supplementary oracle strategies) is necessary to produce trustworthy regression suites.

10 CONCLUSIONS

This thesis addressed the problem of accelerating unit test creation for Python software using large language models (LLMs) in settings where time is limited and specifications are incomplete. Although LLMs can propose tests from code context, their outputs may be mechanically unreliable and, more importantly, semantically uncertain when expected values are not explicitly defined. The objective of this research was to design and evaluate a practical framework that generates `pytest` tests for individual modules, executes them to obtain measurable feedback, iteratively refines failing outputs within a fixed budget, and records artefacts that enable transparent analysis.

The proposed solution is an end-to-end pipeline that performs per-file sandboxing, LLM-driven test generation and refinement, AST-based validation/normalisation, and execution with `Coverage.py` instrumentation. For each evaluated module, it produces raw and refined test files, execution logs, coverage outputs, and structured JSON reports, enabling an artefact-driven evaluation grounded in the `experiments/` directory.

The experimental evidence shows that the framework is effective as a lightweight test bootstrapper for small Python modules. Across eight recorded runs and 22 non-skipped file evaluations, the generated suites achieved high statement coverage, with mean per-file coverage of 93.82% and median coverage of 100% (minimum 83%, maximum 100%). The framework produced strictly passing test suites for 18 out of 22 files (81.8%). When including the operator-supported `marked_complete` state as an accepted completion outcome, 20 out of 22 files (90.9%) reached completion without stalling the overall run. Test suite sizes remained modest (35 raw tests and 36 final tests across 22 files), and refinement was invoked only when needed (4/22 files; mean 0.27 refinement attempts per file).

Qualitative inspection of the stored artefacts clarifies why the framework succeeds and where it remains limited. The pipeline’s guard rails are effective at reducing mechanical failures, as categorised excerpts show no `SyntaxError` and no `ImportError/ModuleNotFound` cases in the recorded runs. However, the dominant residual failures are assertion mismatches in which the LLM encodes an incorrect oracle, particularly for subtle edge cases or underspecified behaviour. This boundary is central to the thesis conclusions: execution feedback and refinement can repair structural issues and improve executability, but they cannot guarantee correct expected values without an independent specification signal.

Overall, this work contributes an offline-capable, reproducible LLM-assisted testing framework for Python and an evidence-based evaluation that connects coverage, executability, refinement

effort, and observed failure modes. The results support a practical interpretation: LLM assistance can efficiently bootstrap runnable tests and improve code exercise, but human review (or stronger oracle strategies) remains necessary to produce trustworthy regression suites.

11 FUTURE WORK

The results of this thesis suggest that LLM-assisted test generation is practical for rapidly producing executable `pytest` suites with high statement coverage on small Python modules when combined with execution feedback and robust normalisation. At the same time, the recorded failure modes show that semantic reliability of assertions remains the dominant barrier to full automation. Future work should therefore prioritise improvements that strengthen oracle quality, broaden evaluation scope, and improve scalability and workflow integration.

At the framework level, several extensions are directly aligned with the current implementation. `Coverage.py` is already executed per file; enabling branch coverage and reporting it alongside statement coverage would provide a stricter adequacy signal with minimal changes. The repository also includes a Cosmic Ray helper, but mutation outcomes are not yet part of the experimental artefacts; integrating mutation testing as an optional stage would better assess fault-detection capability beyond coverage. Scalability can be improved by parallelising independent per-file runs and optimising sandbox and coverage handling for larger repositories.

On the LLM side, the current prompts and post-processing rules enforce executable structure (imports, deduplication, truncated-output salvage, and the one-assert-per-test constraint), yet oracle errors persist when behaviour is subtle or underspecified. A concrete direction is to make refinement more oracle-aware by leveraging execution feedback to propose and select assertions that are consistent with observed behaviour. In addition, the fine-tuning workflow included in this project motivates systematic study of domain-adapted models to reduce empty-test outputs and improve coverage of normal execution paths under local inference constraints.

Finally, the framework would benefit from closer alignment with real development practice. The current CLI and web dashboard could be extended for CI/CD operation (e.g., running on pull requests, generating reviewable test artefacts, and applying configurable quality gates such as passing status and minimum adequacy improvements).

The most important open challenges highlighted by this work are:

- Reliable oracle strategies when specifications are missing or ambiguous.
- Scaling to larger, dependency-heavy codebases without sacrificing reproducibility.
- Balancing diversity with determinism to avoid flaky test suites.
- Metrics that correlate with fault detection, not only coverage.

12 REFERENCES

- [1] C. Pacheco and M. D. Ernst, “Randoop: Feedback-directed random testing for Java,” in *Proceedings of the ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2007, pp. 815–816.
- [2] P. McMinn, “Search-based software test data generation: A survey,” *Software Testing, Verification and Reliability*, vol. 14, no. 2, pp. 105–156, 2004.
- [3] G. Fraser and A. Arcuri, “EvoSuite: Automatic test suite generation for object-oriented software,” in *Proceedings of the Joint European Software Engineering Conference and ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2011, pp. 416–419.
- [4] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, “The oracle problem in software testing: A survey,” *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.
- [5] L. Inozemtseva and R. Holmes, “Coverage is not strongly correlated with test suite effectiveness,” in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2014, pp. 435–445.
- [6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *arXiv preprint arXiv:2005.14165*, 2020.
- [7] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [8] Z. Yuan, Y. Lou, M. Liu, S. Ding, K. Wang, Y. Chen, and X. Peng, “No more manual tests? evaluating and improving ChatGPT for unit test generation,” *arXiv preprint arXiv:2305.04207*, 2023.
- [9] J. W. Duran and S. C. Ntafos, “An evaluation of random testing,” *IEEE Transactions on Software Engineering*, vol. SE-10, no. 4, pp. 438–444, 1984.
- [10] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, “Hints on test data selection: Help for the practicing programmer,” in *Proceedings of the Computer Science and Statistics: 11th Annual Symposium on the Interface*, 1978.

- [11] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.
- [12] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, “Are mutants a valid substitute for real faults in software testing?” in *Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE)*, 2014, pp. 654–665.
- [13] A. Storhaug and J. Li, “Parameter-efficient fine-tuning of large language models for unit test generation: An empirical study,” *arXiv preprint arXiv:2411.02462*, 2024.
- [14] Y. Tang, Z. Liu, Z. Zhou, and X. Luo, “ChatGPT vs SBST: A comparative assessment of unit test suite generation,” *arXiv preprint arXiv:2307.00588*, 2023.
- [15] J. C. King, “Symbolic execution and program testing,” *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, 1976.
- [16] P. Godefroid, N. Klarlund, and K. Sen, “DART: Directed automated random testing,” in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2005, pp. 213–223.
- [17] K. Sen, D. Marinov, and G. Agha, “CUTE: A concolic unit testing engine for C,” in *Proceedings of the Joint European Software Engineering Conference and ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2005, pp. 263–272.
- [18] N. Tillmann and J. de Halleux, “Pex: White box test generation for .NET,” in *Proceedings of the International Conference on Tests and Proofs (TAP)*, 2008, pp. 134–153.
- [19] K. Claessen and J. Hughes, “QuickCheck: A lightweight tool for random testing of Haskell programs,” in *Proceedings of the ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2000, pp. 268–279.
- [20] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, “CodeBERT: A pre-trained model for programming and natural languages,” *arXiv preprint arXiv:2002.08155*, 2020.
- [21] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, “CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” *arXiv preprint arXiv:2109.00859*, 2021.
- [22] L. Yang, C. Yang, S. Gao, W. Wang, B. Wang, Q. Zhu, X. Chu, J. Zhou, G. Liang, Q. Wang, and J. Chen, “On the evaluation of large language models in unit test generation,” *arXiv preprint arXiv:2406.18181*, 2024.

- [23] G. Wang, Q. Xu, L. C. Briand, and K. Liu, “Mutation-guided unit test generation with a large language model,” *arXiv preprint arXiv:2506.02954*, 2025.
- [24] S. Zhou, Z. Zeng, X. Zhou, R. Xie, S. Zhang, and W. Ye, “Seed&Steer: Guiding large language models with compilable prefix and branch signals for unit test generation,” *arXiv preprint arXiv:2507.17271*, 2025.
- [25] D. Liao, X. Yin, S. Pan, C. Ni, Z. Xing, and X. Sun, “Navigating the labyrinth: Path-sensitive unit test generation with large language models,” *arXiv preprint arXiv:2509.23812*, 2025.
- [26] B. P. Miller, L. Fredriksen, and B. So, “An empirical study of the reliability of UNIX utilities,” *Communications of the ACM*, vol. 33, no. 12, pp. 32–44, 1990.
- [27] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, “Prioritizing test cases for regression testing,” *IEEE Transactions on Software Engineering*, vol. 27, no. 10, pp. 929–948, 2001.
- [28] T. Y. Chen, H. Leung, and I. K. Mak, “Adaptive random testing,” in *Proceedings of the 9th Asian Computing Science Conference (ASIAN)*, 2004.
- [29] C. Cadar, V. Ganesh, P. M. Pawlowski, D. L. Dill, and D. R. Engler, “EXE: Automatically generating inputs of death,” in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2006, pp. 322–335.
- [30] C. Cadar, D. Dunbar, and D. R. Engler, “KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs,” in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2008, pp. 209–224.
- [31] P. Godefroid, M. Y. Levin, and D. Molnar, “Automated whitebox fuzz testing,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2008.
- [32] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2011, pp. 283–294.
- [33] S. Segura, G. Fraser, A. B. Sánchez, and A. Ruiz-Cortés, “A survey on metamorphic testing,” *IEEE Transactions on Software Engineering*, 2016.
- [34] V. J. M. Manès, H. Han, C. Han, S. K. Cha, M. Rangamani, E. J. Schwartz, and M. Woo, “The art, science, and engineering of fuzzing: A survey,” *IEEE Transactions on Software Engineering*, 2021.

- [35] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, “CodeGen: An open large language model for code with multi-turn program synthesis,” *arXiv preprint arXiv:2203.13474*, 2022.
- [36] D. Fried, A. Aghajanyan, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “InCoder: A generative model for code infilling and synthesis,” *arXiv preprint arXiv:2204.05999*, 2022.
- [37] Y. Li, D. Choi, J. Chung, N. Duan, E. Francis *et al.*, “Competition-level code generation with AlphaCode,” *Science*, 2022.
- [38] R. Li, L. B. Allal, C. Goutte *et al.*, “StarCoder: May the source be with you!” *arXiv preprint arXiv:2305.06161*, 2023.
- [39] S. Bhatia, T. Gandhi, D. Kumar, and P. Jalote, “Unit test generation using generative AI: A comparative performance analysis of autogeneration tools,” *arXiv preprint arXiv:2312.10622*, 2023.
- [40] A. Deljouyi, R. Koohestani, M. Izadi, and A. Zaidman, “Leveraging large language models for enhancing the understandability of generated unit tests,” *arXiv preprint arXiv:2408.11710*, 2024.
- [41] M. Rodríguez, G. Rossi, and A. Fernandez, “Evaluating large language models for the generation of unit tests with equivalence partitions and boundary values,” *arXiv preprint arXiv:2505.09830*, 2025.

A Prompts Used

This appendix documents the prompt texts used in the implemented LLM-assisted test generation workflow. The prompts are listed in operational order, starting from fine-tuning data formatting and moving to runtime generation and refinement in the pipeline. All prompt blocks are reproduced from the project source files and notebook cells without modification to wording.

A.1 Notebook Prompts for Fine-Tuning and Evaluation

The fine-tuning notebook defines the instruction format used to build training examples and evaluation prompts for the model. The input is Python source code (after notebook preprocessing), and the expected output is raw PyTest test code text.

A.1.1 Notebook Prompt A: Training Text Construction (`make_train_text`)

Purpose: constructs supervised training text by pairing source code with target test code in one instruction-response template. Input: clipped source code and clipped target tests from dataset rows. Expected output: one formatted training sample string.

Instruction:

You are a straightforward expert software testing assistant.
Generate ONLY raw PyTest test cases.
NO explanations. NO comments. Preserve indentation.

Source Code:

{code_ctx}

Required Unit Test Cases:

{target}

A.1.2 Notebook Prompt B: Evaluation Prompt Construction (`make_eval_prompt`)

Purpose: constructs inference-time prompt text for test generation during notebook evaluation. Input: clipped source code only. Expected output: instruction prompt ending at the required test section for model completion.

Instruction:

You are a straightforward expert software testing assistant.

Generate ONLY raw PyTest test cases.
NO explanations. NO comments. Preserve indentation.

Source Code:
{code_ctx}

Required Unit Test Cases:

A.1.3 Notebook Prompt C: System Message Used in Chat Template

Purpose: constrains generated output format during notebook inference and token-masking preparation. Input: prompt text from the notebook templates. Expected output: raw PyTest test file code with no narrative text.

Output ONLY raw PyTest test file code. No explanations. No comments.

A.2 Runtime Prompts Used by the Main Pipeline

The main orchestrated pipeline uses prompt templates in `llm/prompt_templates.py`, called from `llm/test_generator.py` and `llm/test_refiner.py`. These are the primary runtime prompts used during repository processing.

A.2.1 Runtime Prompt 1: Test Generation Template (TEST_GEN_PROMPT)

Purpose: generates initial raw PyTest tests for a sandboxed module. Input: model import path and preprocessed code under test. Expected output: raw test file content containing PyTest-style test functions.

Instruction:

You are a straightforward expert software testing assistant.
Generate ONLY raw PyTest test cases,
NOT unittest or other formats.
Do NOT write test_placeholder tests returning True or empty tests.

MUST GENERATE SEPERATE TEST CASES AND NOT COMBINE MULTIPLE TESTS INTO ONE.
STRICT RULE: each 'test_' function must contain exactly ONE assert statement.

CRITICAL RULE: Stay within the '1200' token generation limit.
If needed, generate fewer tests, but complete the function properly.
Model Import path: {import_path}

Source Code:

{code}

Required Unit Test Cases:

A.2.2 Runtime Prompt 2: Test Refinement Template (TEST_REFINE_PROMPT)

Purpose: refines previously generated tests after execution failures. Input: import path, code under test, existing tests, and pytest failure feedback. Expected output: refined raw PyTest tests that address the reported failures.

Instruction:

You are a straightforward expert software testing REFINER assistant.

Generate ONLY raw REFINED PyTest test cases,

NOT unittest or other formats.

If the existing tests have unittest formats, convert them to PyTest style.

You are given existing tests and the code under test.

Your task is to REFINER the existing tests so they:

- Detect FAULTY imports, FIX Them or delete.
- Remove duplicates
- Improve correctness and edge cases
- Keep or add any imports required for the refined tests
- Do NOT return empty tests
- Keep tests specific to code behavior

MUST GENERATE MORE SEPERATE TEST CASES

and FIX OLD CASES

AND NOT COMBINE MULTIPLE TESTS INTO ONE.

STRICT RULE: each 'test_' function must contain exactly ONE assert statement.

CRITICAL RULE: Stay within the '1200' token limit.

If needed, generate fewer tests, but complete the function properly.

CRITICAL: Never stop in the middle of a test function or a line.

Module import path:

{import_path}

Code under test:

{code}


```
### Existing tests:
```

```
{tests}
```

```
### Pytest failure feedback:
```

```
{pytest_error}
```

```
### Required REFINED Unit Test Cases:
```

A.3 Additional Script-Level System Instructions

The script `pipeline/llm_runner.py` defines explicit system messages passed alongside the above prompt templates. These instruction strings are part of the scripted generation/refinement behavior in that module.

A.3.1 Script System Instruction A: Generation (`LLMRunner._chat`, raw generation call)

Purpose: enforces output style and quantity constraints during generation in `pipeline/llm_runner.py`.

Input: generation prompt text. Expected output: raw PyTest test code, with no prose.

```
Generate ONLY raw PyTest test cases,
```

```
NOT unittest or other formats.
```

```
NO explanations. NO comments. Preserve indentation.
```

```
MUST IMPORT ALL REQUIRED MODULES FOR TEST INSIDE THE TEST CASE CODE.
```

```
*Generate at least 10 UNIQUE test cases if possible.*
```

A.3.2 Script System Instruction B: Refinement (`LLMRunner._chat`, refine call)

Purpose: enforces cleanup and correction behavior during refinement in `pipeline/llm_runner.py`.

Input: refinement prompt text and pytest feedback context. Expected output: refined PyTest test code with corrected imports and reduced duplication.

```
Generate ONLY raw REFINED PyTest test cases,
```

```
NOT unittest or other formats.
```

```
NO explanations. NO comments. Preserve indentation.
```

```
Remove duplicates. Ensure correctness.
```

```
Keep or add any imports required for the refined tests.
```

In summary, the implemented prompting strategy is layered: notebook prompts establish the fine-tuning and evaluation format, runtime templates drive production generation and

refinement, and script-level system instructions constrain response style during chat completion. Together these prompt artifacts define the actual prompt interface used by this project for LLM-assisted test generation.

B Sample Generated Tests

This appendix presents representative test files produced by the implemented LLM-based generation pipeline during a real experiment run (“extracted_20260225_150318”). The samples are reproduced verbatim from the generated artifacts to preserve their original structure and logic. Coverage categories are indicated for each sample as normal-case, edge-case, or error-handling behavior.

B.1 Sample A: Leap-Year Function (Normal and Edge Cases)

Source module: leapyear.py

Generated test file: tests/test_raw_0001.py

Observed role: Parametrized checks over typical and boundary-style calendar years.

```
from eval_sandbox.mod_0001 import *
import pytest

@pytest.mark.parametrize('year, expected', [
    (2000, True),
    (1900, False),
    (2004, True),
    (2001, False),
])
def test_is_leap_year(year, expected):
    assert is_leap_year(year) == expected
```

B.2 Sample B: Factorial Function (Normal and Boundary Input)

Source module: factorial.py

Generated test file: tests/test_raw_0003.py

Observed role: Includes boundary input $n = 0$ and small positive inputs to verify arithmetic progression.

```
from eval_sandbox.mod_0003 import *
import pytest
from eval_sandbox.mod_0003 import get_factorial
```

```
@pytest.mark.parametrize('n,expected', [(0, 1), (1, 1), (2, 2), (3, 6)])
def test_get_factorial(n, expected):
    assert get_factorial(n) == expected
```

B.3 Sample C: Simple Arithmetic (Normal and Error Handling)

Source module: simple.py

Generated test file: tests/test_raw_0004.py

Observed role: Verifies a normal arithmetic path and explicit exception behavior.

```
from eval_sandbox.mod_0004 import *
import pytest

def test_add():
    assert add(2, 3) == 5

def test_divide():
    with pytest.raises(ZeroDivisionError):
        divide(5, 0)
```

B.4 Sample D: Refined Output After Feedback Rounds

Source module: moves2_3.py

Generated test file: tests/test_refined_0002_r2.py

Observed role: Example of a final refined artifact retained after two refinement attempts in the run metadata.

```
from eval_sandbox.mod_0002 import *
import pytest
from eval_sandbox.mod_0002 import calculate_min_moves

def test_calculate_min_moves():
    # Test case 1
    assert calculate_min_moves("..R..L..R.") == 3
    # Test case 2
    assert calculate_min_moves("R...L") == 2
    # Test case 3
    assert calculate_min_moves("....") == 0
```

The selected files demonstrate the practical diversity of generated outputs in the framework: direct assertions for nominal behavior, parametrized boundary checks, explicit exception validation, and retained refined variants.

C Additional Results

This appendix reports supplementary quantitative outcomes from the archived experiment artefacts under `experiments/`. The intent is to provide detail that supports the main evaluation chapter while avoiding repetition of its core narrative. All values in this appendix are taken directly from `summary_runs.json`, `summary_runs.csv`, and `summary_aggregate.json`; no new experiments were performed.

The additional material is organised into three perspectives: (i) run-level completion and test-volume deltas, (ii) refinement and failure-category distributions, and (iii) coverage denominator effects that explain why a fixed number of uncovered statements can yield different coverage percentages across runs.

C.1 Supplementary Run-Level Completion and Test-Volume Detail

Table 13 extends the run-level view by explicitly showing the completion gap between strict pytest passes and completed outcomes (where completed includes `marked_complete`), as well as the difference between final and raw test counts. This information is useful for interpreting operational throughput in runs where strict pass status alone does not capture operator-mediated continuation.

Table 13: Supplementary run-level detail from `experiments/summary_runs.json`.

Run	Files	Pass	Completed	Gap	Raw	Δ Tests
extracted_20260225_143003	2	2	2	0	3	0
extracted_20260225_144837	4	3	3	0	7	0
extracted_20260225_150123	2	2	2	0	3	0
extracted_20260225_150318	4	3	4	1	7	0
extracted_20260225_151314	2	2	2	0	3	0
extracted_20260225_151425	4	3	4	1	7	0
test_project_short_20260225_125217	2	1	1	0	2	1
test_project_short_20260225_142523	2	2	2	0	3	0

The table shows that most runs preserve test volume between raw and final stages. Only one run (`test_project_short_20260225_125217`) shows a net increase in final tests, indicating that refinement can recover from an initially insufficient raw output. The completion gap is non-zero in two four-file runs, reflecting that one file in each run was accepted as `marked_complete` despite not being counted as a strict pass.

C.2 Refinement and Failure-Type Distributions

Whereas the main results chapter reports aggregate percentages, the appendix provides the exact count distributions used to derive those percentages. As shown in Table 14, most file-level evaluations did not require refinement; a small tail required one or two iterations.

Table 14: Distribution of `refine_attempts_used` across all non-skipped file evaluations (`files_count_non_skipped = 22`), from `experiments/summary_aggregate.json`.

Refinement Attempts	Files	Share (%)
0	18	81.8
1	2	9.1
2	2	9.1

This count-level breakdown confirms that refinement remains a selective mechanism rather than the dominant path for obtaining executable outputs in the evaluated set. It also provides a concrete basis for interpreting the observed mean refinement usage of 0.27 attempts per non-skipped file.

Table 15 reports the stored failure-category counts used during aggregate analysis. The classification indicates that assertion-level mismatch is the only explicitly identified failure type in the structured categories, while a small number of failures remain in the “Other/Unknown” group due to heterogeneous report excerpts.

Table 15: Failure-category counts from `failure_categories_counts` in `experiments/summary_aggregate.json`.

Category	Count
AssertionError	1
ImportError/ModuleNotFound	0
SyntaxError	0
TypeError	0
NameError	0
AttributeError	0
ValueError	0
Other/Unknown	3

C.3 Coverage Denominator Sensitivity (Supplementary View)

The main chapter notes that run-level overall coverage frequently appears at either 89% or 94%. Table 16 documents the underlying denominator structure that produces this pattern. The key point is that one missing statement has a larger percentage impact in smaller statement totals.

Table 16: Coverage denominator pattern derived from run-level summaries. “Run count” indicates how many of the eight recorded runs exhibit each statement/missing configuration.

Run count	Files/run	Statements	Missing	Coverage (%)
5	2	9	1	89
3	4	18	1	94

This supplementary table clarifies that the observed difference between 89% and 94% is explained by denominator size rather than by a change in the count of uncovered statements. Consequently, cross-run interpretation should consider statement totals alongside percentages.

No additional figures are introduced in this appendix because the available numerical evidence is fully representable in compact tabular form and can be interpreted directly without visual plotting.